

PROJECT IMPLEMENTATION AND DEFECT REMEDIATION REPORT

Complete Code Explanation, Corrections Applied, Execution Guide and Re-measured Results

Customer Churn Prediction and Retention Analytics

A Nine-Phase Business-Oriented Analytics Pipeline

*Data Cleaning • Exploratory Analysis • Feature Engineering • Segmentation
Predictive Modelling • Explainable AI • Revenue Impact • Interactive Dashboard*

Document	Implementation, remediation and results report
Codebase	customer_churn_100 / customer_churn
Source modules	9 phase scripts + 1 shared feature module + 1 Streamlit application (4 486 lines)
Dataset	Telco Customer Churn — 7 043 records × 21 attributes
Defects corrected	6 issues in two rounds (3 critical defects + 3 follow-up issues) plus 2 consequential fixes
Environment	Dedicated virtual environment (.venv), Python 3.13.13
Best model	Tuned XGBoost — test ROC-AUC 0.8472, recall 0.8262
Pipeline status	All 9 phases re-executed end to end, exit code 0
Report date	18 September 2026 (revision 2)

Table of Contents

- 1** Executive Summary
- 2** Scope, Method and What Changed
- 3** Project Structure and File Inventory
- 4** Virtual Environment and Configuration
- 5** Dataset Description
- 6** Architecture and Data Lineage
- 7** Defect 1 — Hold-out Sample Size
- 8** Defect 2 — Target Information Leakage
- 9** Defect 3 — Unused Phase 4 Derived Features
- 10** Consequential Corrections
- 11** Phase 1 — Data Collection and Understanding
- 12** Phase 2 — Data Cleaning
- 13** Phase 3 — Exploratory Data Analysis
- 14** Phase 4 — Feature Engineering
- 15** Phase 5 — Customer Segmentation
- 16** Phase 6 — Model Development
- 17** Phase 7 — Model Evaluation and Threshold Analysis
- 18** Phase 8 — Churn Probability and Explainable AI
- 19** Phase 9 — Business Recommendations and Revenue Impact
- 20** The Streamlit Dashboard
- 21** Results — Before and After Remediation
- 22** Second-Round Corrections
- 23** Step-by-Step Execution Guide
- 24** Conclusion
- A** Appendix A — Configuration Reference
- B** Appendix B — Complete Artefact Inventory

Every figure, table and numeric value in this report was produced by executing the corrected codebase in the project's virtual environment and reading the resulting files. Results measured before remediation are labelled as such throughout, so the effect of each correction can be traced.

1. Executive Summary

This report documents the customer_churn_100 codebase: a nine-phase churn prediction and retention analytics pipeline with an accompanying Streamlit dashboard. It explains every module at the level of individual code blocks, records the defects that were specified for correction in two rounds, describes exactly how each was fixed, and reports the results measured after the corrected pipeline was re-executed end to end.

Revision 2 adds a second round of corrections: the SVM metric discrepancy between phases 6 and 7, the redundant **ContractRiskFlag** feature, and the collapse of the phase 9 driver cascade into a single category. All three are fixed, the affected phases were re-executed, and every figure in this revision reflects the final code.

1.1 The three defects and their remediation

#	Defect	Correction applied	Verified effect
1	Hold-out sample size. <code>TEST_SIZE = 0.05</code> gave a 353-record test set with 94 churners, while the logs claimed an 80/20 split.	<code>TEST_SIZE = 0.20</code> in phases 6, 7 and 8. The split ratio is now printed from the constant, so the log cannot disagree with the configuration.	Test set 353 → 1 409 records, churners 94 → 374. 95% confidence interval on ROC-AUC ± 0.053 → ± 0.026 .
2	Target information leakage. Phase 5 named clusters using their observed churn rate, then passed that name to phases 6-8 as a predictor.	Segment naming now uses centroid geometry only. Models receive the nominal Cluster code; CustomerSegment is a presentation label.	CustomerSegment no longer appears among the model inputs. Duplicate segment names eliminated.
3	Unused engineered features. Phase 6 read the phase 5 output, so six of the seven phase 4 features never reached a model.	Phase 4 exports unscaled features; phase 5 merges them on customerID with an integrity check.	All engineered features now reach the pipeline; the model matrix is 26 columns after round 2 removed the duplicate flag. TenureBand_0-6 ranks 5th by gain.

Table 1: The three specified defects, the correction applied to each and its verified effect.

1.2 Effect on measured performance

The corrections were not cosmetic. Removing the leaked label while adding six genuine engineered features changed the model materially, and on a test set four times larger the result improved on every ranking metric.

Metric (Tuned XGBoost, hold-out)	Before	After	Change
Test records / churners	353 / 94	1 409 / 374	4.0× larger
ROC-AUC	0.8349	0.8472	+0.0123
Recall	0.7766	0.8262	+0.0496
Precision	0.4932	0.4992	+0.0060
F1 score	0.6033	0.6224	+0.0191
PR-AUC	0.6128	0.6632	+0.0504
Churners missed (false negatives)	21 of 94 (22.3%)	65 of 374 (17.4%)	-4.9 pp
95% CI half-width on ROC-AUC	± 0.053	± 0.026	halved

Table 2: Hold-out performance of the selected model before and after remediation. The 'before' column is the result recorded by the codebase as delivered.

The improvement is genuine, not an artefact of the larger test set.

Removing a leaked feature normally lowers measured performance, because the model loses information it should

never have had. Here ROC-AUC rose instead. The explanation is that defect 2 and defect 3 were fixed together: the leaked segment label was a coarse four-category summary, while the six features recovered by fixing defect 3 carry finer, legitimate signal. The feature importance in Section 18.4 confirms this directly — **TenureBand** and **ValueTier**, both previously discarded, now contribute to the model, while the leaked label is absent entirely. Removing the duplicate **ContractRiskFlag** in round 2 cost nothing: ROC-AUC moved from 0.8470 to 0.8472.

1.3 Pipeline status

All nine phases and the dashboard were executed in a dedicated virtual environment after the corrections. Every phase completed with exit code 0.

- **Phases 1 to 9** re-executed in sequence; all artefacts regenerated from the corrected code.
- **The Streamlit dashboard** starts cleanly and scores uploaded files through the same shared feature module the training phases use, which removes the train/serve mismatch described in Section 10.2.
- **Risk tiering validates cleanly:** customers placed in the High Risk tier show a 78.04% observed churn rate against 9.85% in the Low Risk tier.
- **Mean predicted probability is 0.2652** against an observed base rate of 0.2654, which is strong evidence that the calibration step is working.

1.4 Second-round corrections

Three further issues identified in the first revision have now been corrected. Section 22 gives the evidence for each.

#	Issue	Correction	Verified effect
4	SVM metric discrepancy. Phase 6 labelled with <code>predict()</code> , phase 7 with <code>predict_proba >= 0.50</code> ; for SVC these disagree.	Phase 6 now thresholds <code>predict_proba</code> at an explicit <code>DECISION_THRESHOLD = 0.50</code> .	Phases 6 and 7 report identical metrics for all four shared models (SVM 0.7850 accuracy in both, previously 0.7601 vs 0.7835).
5	Redundant ContractRiskFlag. Identical to <code>Contract == Month-to-month</code> for 100 % of rows.	Excluded from the model matrix; kept in the data as a readable attribute.	Matrix 27 → 26 columns. ROC-AUC 0.8470 → 0.8472 . Contract importance no longer split between two copies.
6	Driver cascade collapse. All 732 targets received the same driver and action.	Contract becomes the shared base offer; each customer's driver is their top SHAP lever, computed in phase 8.	724 targets now span 5 driver categories (333 / 273 / 112 / 4 / 2) instead of 1.

Table 3: The three second-round issues and their verified corrections.

2. Scope, Method and What Changed

2.1 What this report covers

The subject is the code in `customer_churn_100\customer_churn`: nine sequential phase scripts, a Streamlit application, and the shared feature module added during remediation. The report serves three purposes at once, and the sections are ordered so each can be read on its own:

1. **Remediation record (Sections 7 to 10).** What each defect was, why it mattered, the exact code change, and the evidence that it is fixed.
2. **Code documentation (Sections 11 to 20).** Each phase explained block by block, with its inputs, outputs, measured results and generated charts.
3. **Operations (Sections 23 and Appendix A).** Virtual environment setup, the execution sequence, artefact verification and troubleshooting.

2.2 Method

The work proceeded in a fixed order so that every reported number is traceable to an execution of the corrected code:

1. The delivered code and all existing artefacts were read first, and the pre-remediation results recorded as the baseline.
2. A virtual environment was created and the declared dependencies installed, so the corrections were developed and tested against a known set of versions.
3. The three specified defects were corrected, together with two consequential fixes that the corrections made necessary.
4. Every phase was re-executed in sequence, in the virtual environment, from the raw CSV through to the business recommendations.
5. The dashboard was started and its scoring path exercised against the retrained model to confirm the serving path matches the training path.
6. All figures and tables in this report were then read from the regenerated artefacts.

2.3 Files changed

File	Change	Reason
<code>churn_features.py</code>	New module	Single definition of every engineered feature and of the segment naming rule, imported by phase 4, phase 5 and the dashboard so training and serving cannot drift apart. In round 2 it also excludes <code>ContractRiskFlag</code> from the model (issue 5) and defines the SHAP lever groups (issue 6)
<code>phase4_feature_engineering.py</code>	Modified	Builds features through the shared module; exports unscaled features and persists the ARPU quartile edges (defect 3)
<code>phase5_customer_segmentation.py</code>	Modified	Centroid-based segment naming (defect 2); merges the phase 4 features (defect 3); persists the fitted segmentation model
<code>phase6_model_development.py</code>	Modified	<code>TEST_SIZE = 0.20</code> (defect 1); model matrix built by the shared helper (defect 2); labels from <code>predict_proba</code> at <code>DECISION_THRESHOLD</code> (issue 4)
<code>phase7_model_evaluation.py</code>	Modified	<code>TEST_SIZE = 0.20</code> (defect 1); model matrix built by the shared helper (defect 2)
<code>phase8_churn_probability_xai.py</code>	Modified	<code>TEST_SIZE = 0.20</code> (defect 1); model matrix built by the shared helper, including the SHAP reason-code matrix (defect 2); exports per-customer SHAP levers (issue 6)

phase9_business_recommendations.py	Modified	Reads recall and the baseline churn rate from upstream artefacts; driver and action taken from the phase 8 SHAP levers instead of a rule cascade (issue 6)
app/app.py	Modified	Scores through the shared module and the persisted segmentation model instead of a hand-written segment heuristic
Phases 1, 2, 3	Unchanged	No defect was identified in data profiling, cleaning or exploratory analysis

Table 4: Every file changed during remediation, with the reason for each change.

Why a shared module was introduced rather than editing each script in place.

Defect 3 requires the engineered features to reach the models, and the dashboard must then build those same features for an uploaded customer. Duplicating the construction in three places is how train/serve mismatches begin — and the delivered app already showed one, reconstructing segments from a hand-written rule that did not reproduce the trained clusters. A single imported definition makes the mismatch structurally impossible. The module contains no new modelling logic; it is the existing feature code, moved to one place and given fit/transform semantics so training boundaries are reused at prediction time.

3. Project Structure and File Inventory

All scripts use paths relative to the working directory, so every module must be executed with `customer_churn_100\customer_churn` as the current directory.

3.1 Directory tree

customer_churn_100/ └─ customer_churn/				<- working directory for all scripts
├─ churn_features.py	NEW	shared feature + segmentation logic		
├─ phase1_data_understanding.py	97 lines	data profiling		
├─ phase2_data_cleaning.py	160 lines	cleaning and validation		
├─ phase3_eda.py	317 lines	exploratory analysis		
├─ phase4_feature_engineering.py	235 lines	feature construction		
├─ phase5_customer_segmentation.py	361 lines	K-Means segmentation		
├─ phase6_model_development.py	539 lines	training and tuning		
├─ phase7_model_evaluation.py	580 lines	comparison, thresholds		
├─ phase8_churn_probability_xai.py	541 lines	calibration, SHAP		
├─ phase9_business_recommendations.py	487 lines	revenue impact		
├─ app/				
│ └─ app.py	1 145 lines	Streamlit dashboard		
│ └─ README.md				
├─ data/	raw, cleaned, features, segmented + 8 result files			
├─ charts/	24 generated PNG figures			
├─ models/	5 serialised artefacts			
├─ requirements.txt				
└─ .venv/	NEW	virtual environment		

Listing 1: Directory layout after remediation. Items marked NEW were added during this work.

3.2 Module inventory

Module	LOC	Reads	Writes
churn_features.py	~270	nothing (library)	nothing (library)
phase1_data_understanding.py	97	Customer-Churn.csv	console report only
phase2_data_cleaning.py	160	Customer-Churn.csv	Customer-Churn-Cleaned.csv
phase3_eda.py	317	Customer-Churn-Cleaned.csv	9 charts
phase4_feature_engineering.py	235	Customer-Churn-Cleaned.csv	Features-Unscaled.csv, Model-Ready.csv, feature artefacts
phase5_customer_segmentation.py	361	Cleaned.csv + Features-Unscaled.csv	Segmented.csv, segmentation model, 4 charts
phase6_model_development.py	539	Customer-Churn-Segmented.csv	best_churn_model.joblib, performance CSV, 3 charts
phase7_model_evaluation.py	580	Customer-Churn-Segmented.csv	evaluation CSV, threshold CSV, 5 charts
phase8_churn_probability_xai.py	541	Customer-Churn-Segmented.csv	risk predictions, importances, reason codes, calibrated model, 4 charts
phase9_business_recommendations.py	487	phase 8 predictions + phase 7 results	retention targets, financial summary, mapping, 3 charts
app/app.py	1 145	every artefact above	scored CSV downloads

Table 5: Module inventory showing size and file dependencies.

Execution order is mandatory, and phase 5 now depends on phase 4.

Each script reads a file the previous one wrote. The remediation added one new dependency: phase 5 merges the

unscaled features that phase 4 exports, so phase 4 must run first. Phase 5 checks for that file and exits with a clear message rather than silently producing a dataset with the engineered features missing — which is precisely how defect 3 went unnoticed.

4. Virtual Environment and Configuration

The project declares its dependencies in requirements.txt but shipped without an environment. A dedicated virtual environment was created for this work, and every result in this report was produced inside it. This matters for more than tidiness: the serialised models in models/ are pickled objects whose compatibility depends on the scikit-learn version that wrote them, so pinning the environment is what makes the artefacts reloadable.

4.1 Environment creation

```
REM Every script uses relative paths, so this must be the working directory.
cd "C:\Users\user\Desktop\Customer churn prediction\customer_churn_100\customer_churn"

REM Create and activate the environment
python -m venv .venv
.venv\Scripts\activate

REM Install the declared dependencies
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

REM Confirm every import resolves before running any phase
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, xgboost, shap, joblib, streamlit;
print('environment ok')"
```

Listing 2: Creating the virtual environment and installing dependencies.

4.2 Verified versions

These are the versions actually installed and used to produce every result in this report.

Component	Version	Used by	Role
Python	3.13.13	all phases	64-bit CPython
pandas	3.0.5	all phases	data frames, grouping, merging
NumPy	2.5.3	phases 5-9	array backend
scikit-learn	1.9.1	phases 4-9	pipelines, K-Means, calibration, metrics
XGBoost	3.4.1	phases 6, 7, 8	gradient-boosted classifier
SHAP	0.52.0	phase 8	global and local explanations
matplotlib	3.11.1	phases 3, 5-9	figure rendering
seaborn	0.13.2	phases 3, 5-9	statistical plot styling
joblib	1.6.0	phases 4, 5, 8	artefact serialisation
Streamlit	1.63.0	dashboard	web application runtime

Table 6: Verified runtime environment. All nine phases executed successfully against these versions.

Both environment problems reported against the previous codebase are now resolved.

The earlier review recorded that xgboost was absent from the interpreter and that the serialised models raised InconsistentVersionWarning when loaded. Installing the declared requirements into a dedicated environment fixes the first, and re-executing every phase regenerates all five model artefacts under scikit-learn 1.9.1, which fixes the second. The models in models/ now load without warnings.

4.3 Configuration constants that changed

Constant	Declared in	Before	After	Effect
TEST_SIZE	phases 6, 7, 8	0.05	0.20	hold-out grows from 353 to 1409 records
MODEL_RECALL	phase 9	0.7766 hard-coded	read from phase 7	financial model always quotes the recall actually measured
BASELINE_CHURN_RATE	phase 9	0.2654 hard-coded	measured from data	cannot drift from the dataset
MODEL_EXCLUDED_COLUMNS	churn_features.py	Cluster dropped, segment kept	segment and ContractRiskFlag dropped, Cluster kept	removes the leaked label and the duplicate flag
DECISION_THRESHOLD	phase 6	implicit (predict())	0.50 explicit	phase 6 and phase 7 label identically
RANDOM_STATE	phases 5-8	42	42 (unchanged)	reproducibility of split, clustering and every model
OPTIMAL_K	phase 5	4	4 (unchanged)	number of behavioural segments

Table 7: Configuration constants altered by the remediation.

5. Dataset Description

The pipeline is built on the Telco Customer Churn dataset: 7 043 customer records described by 21 attributes — one identifier, nineteen predictors and one binary target. The data was not modified during remediation, so the descriptive statistics below are unchanged from the delivered project.

5.1 Attribute schema

Group	Attributes	Role
Identifier	<code>customerID</code>	excluded from every model matrix
Demographic	<code>gender</code> , <code>SeniorCitizen</code> , <code>Partner</code> , <code>Dependents</code>	who the customer is
Account	<code>tenure</code> , <code>Contract</code> , <code>PaperlessBilling</code> , <code>PaymentMethod</code>	commercial relationship and billing behaviour
Core service	<code>PhoneService</code> , <code>MultipleLines</code> , <code>InternetService</code>	primary products subscribed
Add-on service	<code>OnlineSecurity</code> , <code>OnlineBackup</code> , <code>DeviceProtection</code> , <code>TechSupport</code> , <code>StreamingTV</code> , <code>StreamingMovies</code>	value-added product adoption
Financial	<code>MonthlyCharges</code> , <code>TotalCharges</code>	recurring and lifetime revenue
Target	<code>Churn</code>	Yes = customer left, No = customer retained

Table 8: Attribute schema of the source dataset, grouped by analytical role.

5.2 Target distribution

Class	Records	Share	Meaning
No (retained)	5 174	73.46 %	customer still active
Yes (churned)	1 869	26.54 %	customer terminated the service
Total	7 043	100.00 %	complete customer base

Table 9: Class distribution of the target variable.

The 26.54 % positive rate is what makes accuracy misleading here. A model predicting "no churn" for everyone scores 73.46 % accuracy while identifying not one leaving customer. Every classifier in the study is therefore instantiated with `class_weight="balanced"` or an equivalent `scale_pos_weight` correction, and models are selected on ROC-AUC rather than accuracy.

5.3 Data-quality issues in the raw file

Issue	Extent	Resolution
<code>TotalCharges</code> stored as text	all 7 043 rows	cast with <code>pd.to_numeric(errors="coerce")</code> in phase 2
Blank <code>TotalCharges</code>	11 rows, all with <code>tenure</code> = 0	set to 0.0 — a customer billed nothing has accumulated nothing
Redundant service labels	7 columns	"No internet service" and "No phone service" folded into "No"
Duplicate records	0 found	no action required
Explicit nulls	0 found	no general imputation required

Table 10: Data-quality issues and how the pipeline resolves them.

6. Architecture and Data Lineage

The implementation is a linear nine-stage pipeline. Each stage is a standalone script that reads a file, performs one class of work and writes its output back to disk before terminating. There is no orchestration layer and no in-memory hand-off between stages.

Because every intermediate result is materialised, any stage can be re-executed without re-running those before it, and every intermediate state is inspectable on disk. That is convenient — but it is also how defect 3 survived: a file can be written by one phase and quietly ignored by the next, and nothing in a file-based pipeline signals that.

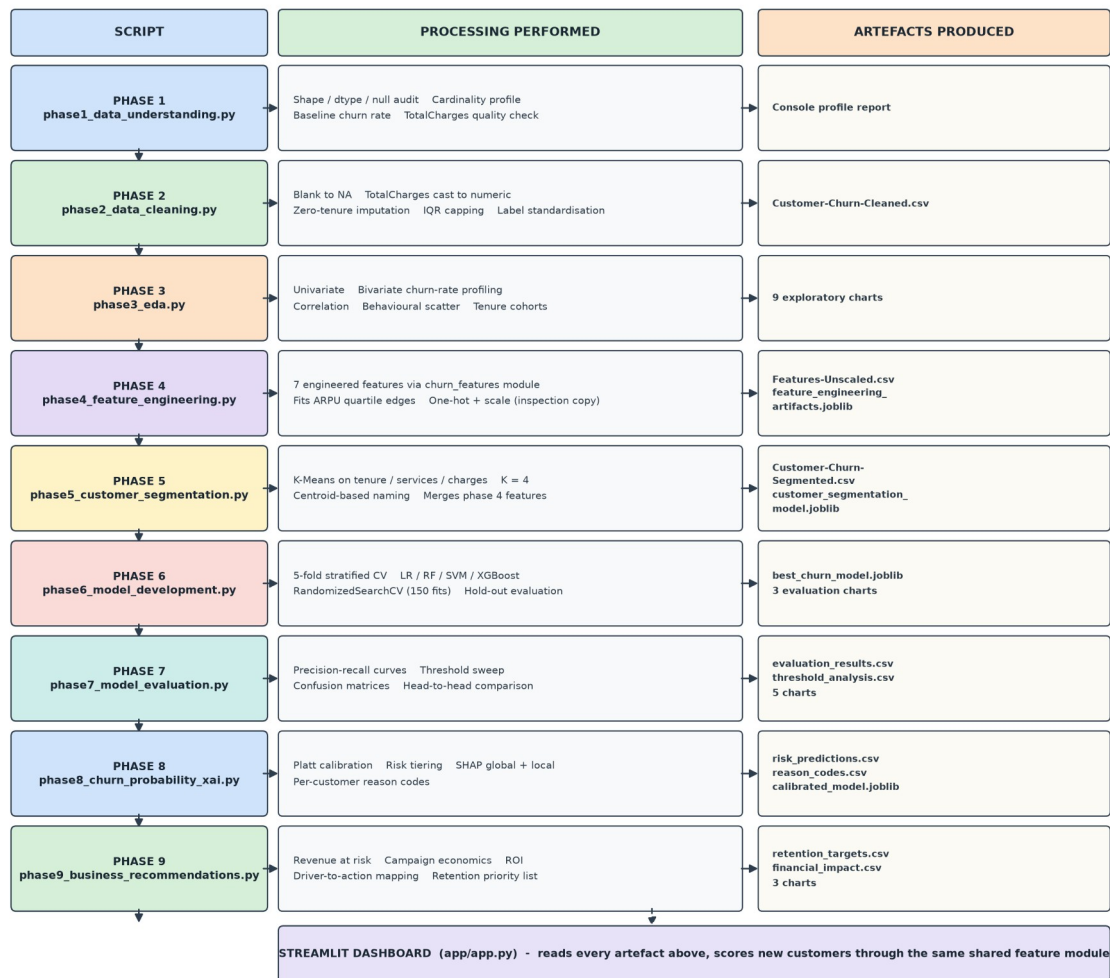


Figure 1: The nine-phase pipeline. Each row shows the script, the processing it performs and the artefacts it writes. The dashboard reads every artefact and scores new customers through the same shared feature module.

6.1 Corrected data lineage

The diagram below traces each dataset from the raw file to the model layer after remediation. The two structural changes are the merge arrow from phase 4 into the segmented dataset, and the model matrix now being built by a single shared helper that both the training phases and the dashboard call.

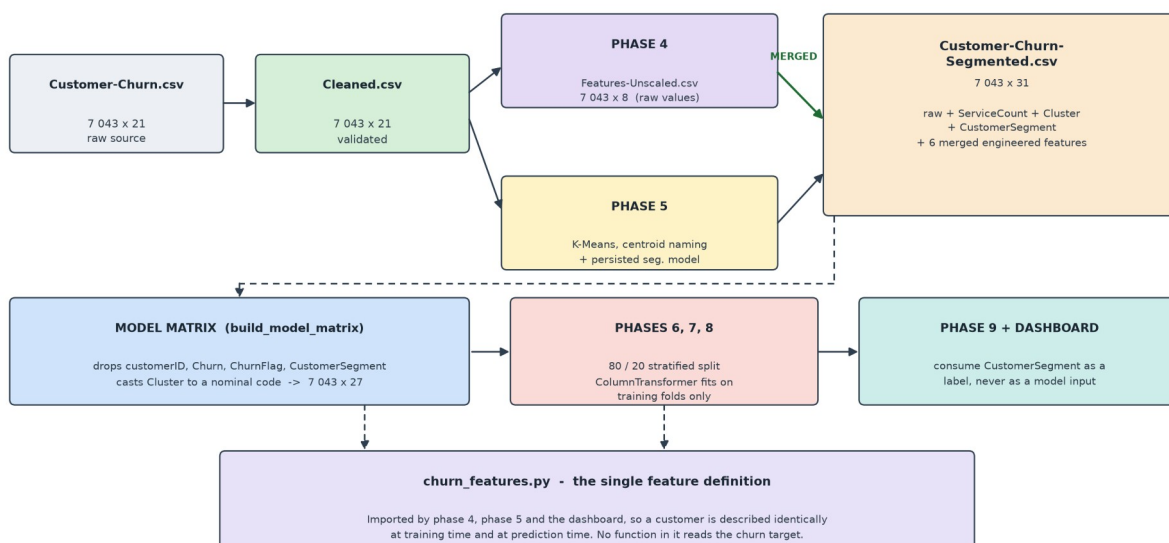


Figure 2: Data lineage after remediation. The phase 4 output is now merged into the segmented dataset rather than being written and ignored.

6.2 Stage responsibilities and runtime

Stage	Responsibility	Measured runtime
Phase 1	Profile the raw file and quantify data-quality defects	under 2 s
Phase 2	Repair, validate and standardise	under 3 s
Phase 3	Quantify churn drivers and write 9 charts	15-30 s
Phase 4	Build 7 features, fit tier edges, export unscaled copy	under 5 s
Phase 5	Cluster, name from centroids, merge features, persist model	20-40 s
Phase 6	Train 5 models, 150-fit randomized search, evaluate	about 4 minutes
Phase 7	Re-fit 4 models, threshold sweep, 5 charts	about 3 minutes
Phase 8	Calibrate, tier, SHAP global and local	about 1 minute
Phase 9	Revenue at risk, campaign economics, 3 charts	under 10 s

Table 11: Responsibility and measured runtime of each stage in the virtual environment.

Phases 6 and 7 dominate the runtime. Phase 6 performs 150 model fits in its randomized search alone, and both phases configure Random Forest and baseline XGBoost with 2 000 estimators.

7. Defect 1 — Hold-out Sample Size

7.1 The defect

Phases 6, 7 and 8 each declared `TEST_SIZE = 0.05`, producing a hold-out set of 353 records containing 94 churners. Two printed lines and one summary line nevertheless described the split as 80/20.

```
TEST_SIZE = 0.05                                # actual configuration

# 2. Stratified 80/20 train-test split            # comment says otherwise
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE)
...
print("- Applied a stratified 80/20 train-test split.") # contradicts the constant
```

Listing 3: The defect as delivered — the constant and the reported split disagreed.

7.2 Why it mattered

The problem was never a wrong number; it was an unstable one. With 94 positive and 259 negative cases, the Hanley-McNeil standard error of the area under the ROC curve is 0.0273, giving a 95 % confidence interval of roughly ± 0.053 . The top three models in the delivered results were separated by 0.0020 of ROC-AUC — about one twenty-fifth of that interval. The published ranking could not be distinguished from sampling noise.

Quantity	Before (5 %)	After (20 %)	Reading
Test records	353	1 409	4.0× more evidence
Churners in test set	94	374	the minority class drives the interval width
Hanley-McNeil SE of ROC-AUC	0.0273	0.0133	halved
95 % CI half-width	± 0.053	± 0.026	differences of 0.03 are now resolvable
Gap between top 2 models	0.0002	0.0024	still inside the interval — see the note below

Table 12: Statistical effect of enlarging the hold-out partition.

7.3 The correction

```
# A 5% hold-out left 353 records with only 94 churners, an interval on
# ROC-AUC of roughly +/-0.045 -- wider than the gap between the top models,
# so they could not be ranked. 20% gives 1409 records and ~374 churners.
TEST_SIZE = 0.20

# The split ratio is now derived from the constant, so the printed
# description cannot drift away from the actual configuration.
print(f"Split ratio: {1 - TEST_SIZE:.0%} train / {TEST_SIZE:.0%} test")
print(f"Training set: {X_train.shape[0]} records")
print(f"Test set: {X_test.shape[0]} records")
print(f"Churners in test set: {int(y_test.sum())}")
```

Listing 4: The correction, applied identically in phases 6, 7 and 8.

Printing the ratio from the constant is the part that prevents the defect recurring. The original discrepancy arose because a human-written string described a configuration that later changed; deriving the description from the value removes the possibility.

Verified output

```
2. STRATIFIED TRAIN-TEST SPLIT
```

```
Split ratio: 80% train / 20% test
Training set: 5634 records
Test set: 1409 records
Churners in test set: 374
Training churn rate: 26.54%
Test churn rate: 26.54%
```

Listing 5: Phase 6 console output after the correction. Stratification holds the churn rate at 26.54 % in both partitions.

An honest qualification on model ranking.

Enlarging the hold-out narrows the interval but does not, on this data, make the top models separable. Tuned XGBoost leads Logistic Regression by 0.0024 of ROC-AUC against a 95 % half-width of 0.026. The correct reading is still that the leading models perform equivalently and that the choice between them should rest on other grounds — interpretability, training cost, operating point. What the correction buys is the ability to state that conclusion with evidence, rather than publishing a ranking the data never supported.

8. Defect 2 — Target Information Leakage

8.1 The defect

This was the most serious of the three, because it affected the credibility of a published result rather than merely wasting work or widening an interval.

Phase 5 clusters customers on tenure, service count and monthly charges. That part is clean — K-Means never sees the churn label. The naming step that followed was not:

```
churn_median = cluster_profile["ChurnRate"].median()      # <- reads the target

for cluster_id, values in profile.iterrows():
    tenure = values["MeanTenure"]
    charges = values["MeanMonthlyCharges"]
    churn = values["ChurnRate"]                          # <- reads the target

    if tenure >= tenure_median and charges >= charge_median and churn < churn_median:
        segment_names[cluster_id] = "High-Value Loyal"
    elif tenure >= tenure_median and charges >= charge_median and churn >= churn_median:
        segment_names[cluster_id] = "High-Value At-Risk"
    elif tenure < tenure_median and churn >= churn_median:
        segment_names[cluster_id] = "New & Fragile"
    else:
        segment_names[cluster_id] = "Low-Value Price-Sensitive"
```

Listing 6: The leaking naming rule as delivered. Each cluster's observed churn rate, computed over the entire dataset including the test rows, selected its label.

Phases 6, 7 and 8 then dropped the numeric **Cluster** column but kept **CustomerSegment** as a categorical predictor. In the delivered model it ranked as the second most important feature by gain. A label chosen by consulting the outcome was being used to predict that outcome.

8.2 Why it mattered

- **The reported score was optimistic by an unknown margin.** Test-row churn contributed to the median that selected each label, so information from the hold-out partition reached the training matrix.
- **The label could not be reproduced at prediction time.** A new customer has no observed churn rate, so the training-time definition of the feature simply does not exist in production.
- **It caused a real train/serve mismatch.** The dashboard worked around the impossibility with a hand-written rule on tenure and charges. That rule did not reproduce the trained clusters, so an uploaded customer could be scored with a segment the model had never associated with their behaviour. Section 10.2 covers this.

8.3 The correction

Naming now depends only on where a cluster's centroid sits, using attributes known for any customer at prediction time. Churn rate is still reported for each cluster — it is an output of the analysis — but it is no longer an input to the label.

```
def name_segments_from_centroids(centroids):
    """Name each cluster from where its centroid sits, never from its churn rate."""
    tenure_median = centroids["tenure"].median()
    charge_median = centroids["MonthlyCharges"].median()

    names = {}
    for cluster_id, row in centroids.iterrows():
        established = row["tenure"] >= tenure_median
```

```

high_spend = row["MonthlyCharges"] >= charge_median

if established and high_spend:         names[cluster_id] = "Established High-Value"
elif established and not high_spend:   names[cluster_id] = "Established Budget"
elif not established and high_spend:  names[cluster_id] = "New High-Spend"
else:                                 names[cluster_id] = "New Budget"
...

```

Listing 7: The corrected naming rule, in `churn_features.py`. No branch reads the target.

The second half of the correction removes the label from the model matrix altogether. Because segment names are now a deterministic function of cluster geometry, **Cluster** and **CustomerSegment** carry identical information; passing both would simply duplicate it. The models receive **Cluster**, cast to a nominal code so the encoder does not treat it as an ordinal quantity:

```

MODEL_EXCLUDED_COLUMNS = ["customerID", "Churn", "ChurnFlag",
                           "CustomerSegment"]

def build_model_matrix(df):
    X = df.drop(columns=[c for c in MODEL_EXCLUDED_COLUMNS if c in df.columns])
    return to_nominal_cluster(X)      # "Cluster_0", "Cluster_1", ...

```

Listing 8: The model matrix helper. **Cluster** is cast to a string because, left as an integer, it would be scaled as though cluster 3 were three times cluster 1.

8.4 Verification

The corrected naming produces four distinct, geometry-derived segments. Churn rate is shown here as a measured property of each segment, not as an input to its name — and the segments still separate churn strongly, from 4.44 % to 47.37 %, which is the evidence that behavioural clustering finds real structure without being told the answer.

```

4. SEGMENT NAMING (CENTROID-BASED, TARGET-FREE)
Naming inputs: mean tenure and mean monthly charges per cluster.
Churn rate is NOT used, so the label carries no target information.
Cluster 0: New Budget          (tenure=10.16, monthly=29.58, services=0.30)
Cluster 1: Established High-Value (tenure=56.89, monthly=90.59, services=4.16)
Cluster 2: Established Budget   (tenure=54.54, monthly=31.81, services=0.58)
Cluster 3: New High-Spend      (tenure=15.05, monthly=77.58, services=1.83)

```

Listing 9: Phase 5 console output after the correction.

Check	Result
ChurnRate referenced in the naming rule	No — removed
CustomerSegment present in the model matrix	No — excluded
Cluster present in the model matrix	Yes, as a nominal code (Cluster_0 ... Cluster_3)
CustomerSegment in the final feature-importance ranking	Absent — it is no longer a model input
Duplicate segment names	None — the delivered code produced "New & Fragile" and "New & Fragile 2"
Segment still available to phase 9 and the dashboard	Yes — carried through as a presentation label

Table 13: Verification checks confirming the leakage is removed while the business label remains available for reporting.

The duplicate-name defect was fixed as a side effect.

The delivered rule classified two very different clusters identically, and a de-duplication branch appended a numeral, yielding "New & Fragile" and "New & Fragile 2". Those two clusters differ sharply — 29.58 against 77.58 in mean monthly charges, and 24.19 % against 47.37 % churn — so the shared root name obscured exactly the distinction the retention strategy depends on. The centroid rule separates them by construction, as "New Budget" and "New High-Spend".

9. Defect 3 — Unused Phase 4 Derived Features

9.1 The defect

Phase 4 constructed seven engineered features, one-hot encoded every categorical column, standardised every numeric column and wrote a 36-column model-ready dataset. Phase 6 then opened Customer-Churn-Segmented.csv instead — the phase 5 output, built from the cleaned file. Phases 7 and 8 did the same.

The result was that ARPU, ChargeRatio, TenureBand, ContractRiskFlag, ValueTier and AutoPayFlag never reached any model. Only ServiceCount survived, and only because phase 5 happened to recompute it independently. Six of the seven engineered features — and with them the entire analytical justification for phase 4 — were computed, written to disk and discarded.

9.2 Why a direct merge of the phase 4 output would have been wrong

The obvious fix is to point phase 6 at Customer-Churn-Model-Ready.csv. That would introduce a subtler problem. Phase 4 scales its output with a `StandardScaler` fitted on the entire dataset, whereas phases 6 to 8 fit their own scaler inside a `ColumnTransformer` that sees only the training rows of each cross-validation fold. Feeding them pre-scaled columns would both scale the data twice and leak the test distribution into training through the fitted means and standard deviations — trading one leakage defect for another.

The correction therefore moves **unscaled** features, leaving all scaling to the existing leakage-safe pipeline.

9.3 The correction

Phase 4 exports raw feature values

```
unscaled_features = df[[ID_COLUMN] + ENGINEERED_FEATURE_COLUMNS].copy()
unscaled_features.to_csv(FEATURES_FILE, index=False)

print(f"Rows exported: {len(unscaled_features)}")
print(f"Feature columns: {ENGINEERED_FEATURE_COLUMNS}")
```

Listing 10: Phase 4 writes Customer-Churn-Features-Unscaled.csv alongside its existing output.

Phase 5 merges them, and refuses to proceed silently if they are missing

```
if os.path.exists(FEATURES_FILE):
    engineered = pd.read_csv(FEATURES_FILE)

    # ServiceCount already exists on df and is identical by construction.
    merge_columns = ["customerID"] + [
        column for column in ENGINEERED_FEATURE_COLUMNS
        if column != "ServiceCount"
    ]

    df = df.merge(engineered[merge_columns], on="customerID", how="left")

    unmatched = df[merge_columns[1]].isna().sum()
    print(f"Rows without a matching engineered record: {unmatched}")

    if unmatched > 0:
        raise SystemExit("Engineered features are missing for some customers. "
                        "Re-run phase 4 before phase 5.")
    else:
        raise SystemExit(f"Required input not found: {FEATURES_FILE}")
```

Listing 11: The merge in phase 5, with an integrity check. A silent partial merge would reproduce defect 3 in a harder-to-detect form.

The explicit failure paths matter more than the merge itself. Defect 3 persisted because a missing feature is invisible — the pipeline runs, the models train, the metrics look plausible. Failing loudly on a missing file or an unmatched row converts a silent degradation into an obvious error.

9.4 Verification

```
7. ENGINEERED FEATURE MERGE
Merged from: data/Customer-Churn-Features-Unscaled.csv
Columns added: 6
Features now available to phases 6-8: ['TenureBand', 'ARPU', 'ChargeRatio',
                                       'ContractRiskFlag', 'ValueTier', 'AutoPayFlag']
Rows without a matching engineered record: 0

8. OUTPUT
Segmented dataset saved successfully: data/Customer-Churn-Segmented.csv
Final shape: (7043, 31)
```

Listing 12: Phase 5 console output confirming the merge.

Feature	Reached a model before?	Reaches a model now?	Rank in final importance
TenureBand	No	Yes	5th (0.0337, 0-6 months band)
ValueTier	No	Yes	17th (0.0124, Medium tier)
ARPU	No	Yes	outside top 20
ChargeRatio	No	Yes	outside top 20
AutoPayFlag	No	Yes	outside top 20; feeds the Billing lever
ServiceCount	Yes (recomputed by phase 5)	Yes	outside top 20; feeds the Engagement lever
ContractRiskFlag	No	No — excluded in round 2	duplicate of Contract (Section 22.2)

Table 14: Each engineered feature before and after the merge, with its position in the phase 8 gain-importance ranking of the final model.

The recovered features are doing real work — with one exception.

TenureBand now ranks 5th by gain and **ValueTier** appears in the top 20; before the correction neither could influence a prediction. **AutoPayFlag** and **ServiceCount** also feed the per-customer SHAP levers that phase 9 uses to choose retention actions. The exception is **ContractRiskFlag**. In the first revision it ranked first by gain, but it equals `Contract == Month-to-month` for every row, so its rank came from splitting the contract signal with an identical column. Round 2 excluded it from the model with no loss in ROC-AUC.

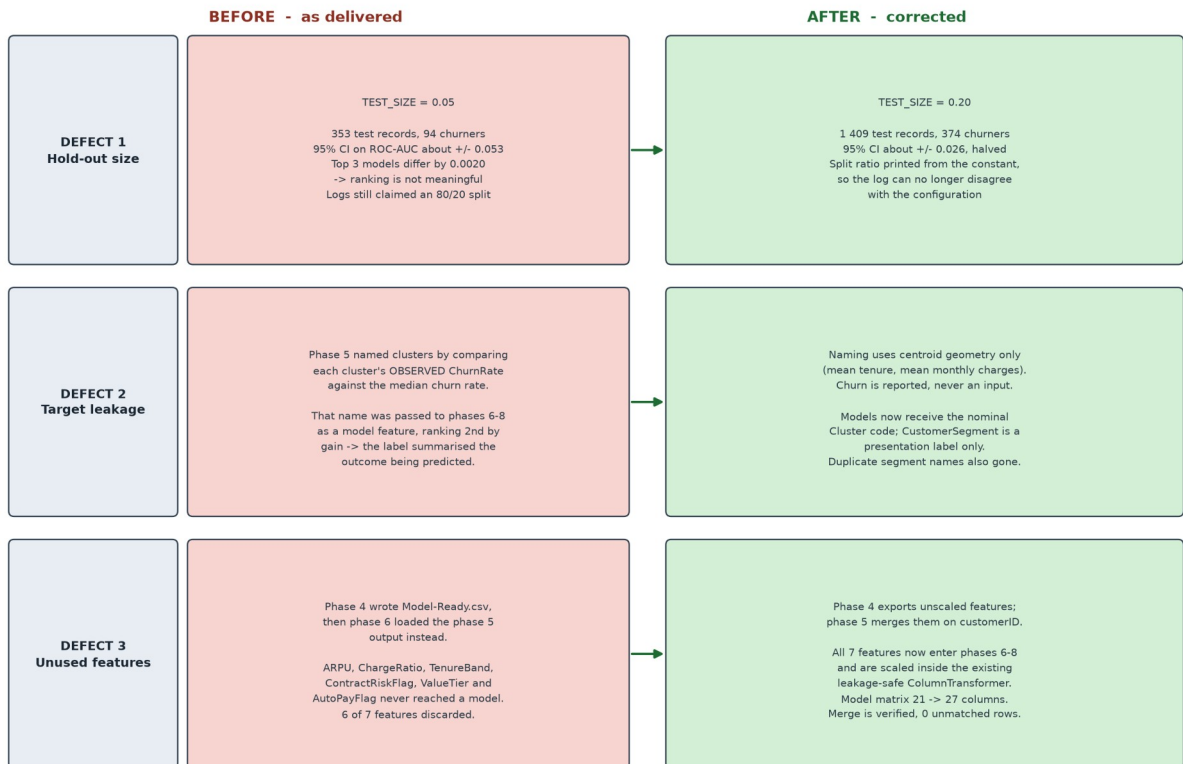


Figure 3: The three defects and their corrections, side by side.

10. Consequential Corrections

Two further changes were required, not because they were on the remediation list, but because correcting the three specified defects would otherwise have left the project inconsistent. Both are direct consequences of the main fixes.

10.1 Phase 9 hard-coded the model's recall

Phase 9 builds the entire financial case on the model's recall, and that value was written into the file as a literal:

```
MODEL_RECALL = 0.7766          # the recall measured on the old 5% hold-out
BASELINE_CHURN_RATE = 0.2654
```

Listing 13: The constants as delivered.

Changing `TEST_SIZE` changes recall. Had this constant been left alone, phase 9 would have continued reporting a campaign built on 0.7766 while the model it describes actually achieves 0.8262 — a financial projection quietly describing a classifier that no longer exists. The constant is now read from the upstream artefact:

```
if os.path.exists(EVALUATION_FILE):
    evaluation = pd.read_csv(EVALUATION_FILE)
    best_row = evaluation.sort_values("ROC_AUC", ascending=False).iloc[0]

    MODEL_RECALL = float(best_row["Recall"])
    BEST_MODEL_NAME = str(best_row["Model"])
    recall_source = f"phase 7 hold-out result for {BEST_MODEL_NAME}"
else:
    MODEL_RECALL = FALLBACK_MODEL_RECALL
    recall_source = "fallback constant (phase 7 results not found)"

if os.path.exists(CLEANED_FILE):
    cleaned = pd.read_csv(CLEANED_FILE, usecols=["Churn"])
    BASELINE_CHURN_RATE = float((cleaned["Churn"] == "Yes").mean())
```

Listing 14: Recall and the baseline rate are now derived, with the source printed so a reader can see which was used.

```
1. INPUT DATA
Customers loaded: 7043
Model recall: 0.8262 (phase 7 hold-out result for Tuned XGBoost)
Baseline churn rate: 0.2654 (measured on the cleaned dataset)
```

Listing 15: Phase 9 now states where each figure came from.

10.2 The dashboard reconstructed segments with a different rule

This was a genuine train/serve mismatch, and it existed because of defect 2. Since the training-time segment label was defined using churn — which an incoming customer does not have — the dashboard could not reproduce it, so it approximated with a hand-written rule:

```
def assign_customer_segment(dataframe):          # as delivered
    def segment_customer(row):
        if row["tenure"] <= 20 and row["MonthlyCharges"] >= 65:
            return "New & Fragile 2"
        if row["tenure"] <= 20 and row["MonthlyCharges"] < 65:
            return "New & Fragile"
        if row["tenure"] >= 45 and row["MonthlyCharges"] >= 65 and \
            row["ServiceCount"] >= 2:
            return "High-Value Loyal"
        return "Low-Value Price-Sensitive"
```

Listing 16: The dashboard's approximation of a feature it could not reproduce.

Hard thresholds at tenure 20 and charges 65 do not reproduce K-Means boundaries in a standardised three-dimensional space. An uploaded customer could therefore be scored carrying a segment the model had never associated with their behaviour.

The correction persists the fitted segmentation model in phase 5 and has the dashboard use it:

```
# phase 5 -- persist what was actually fitted
segmentation_artifacts = {
    "scaler": scaler,
    "kmeans": final_kmeans,
    "segment_names": segment_names,
    "segmentation_columns": SEGMENTATION_COLUMNS,
    "optimal_k": OPTIMAL_K,
}
joblib.dump(segmentation_artifacts, SEGMENTATION_MODEL_FILE)

# churn_features.py -- assign clusters to new data with the trained centroids
def assign_clusters(df, segmentation_artifacts):
    scaled = segmentation_artifacts["scaler"].transform(df[SEGMENTATION_COLUMNS])
    df["Cluster"] = segmentation_artifacts["kmeans"].predict(scaled)
    df["CustomerSegment"] = df["Cluster"].map(segmentation_artifacts["segment_names"])
    return df
```

Listing 17: The corrected serving path uses the trained model rather than a rule of thumb.

The same principle applies to **ValueTier**, whose quartile boundaries are learned from the training distribution. Phase 4 now persists those edges and the dashboard reuses them, so an uploaded customer is placed in the tier the training data defines rather than a tier recomputed from whatever batch they happen to arrive in.

10.3 Verification of the serving path

The dashboard was started and its scoring path exercised against the retrained model. Every model input is reconstructed from raw columns with no gaps.

```
Model expects 26 features | ContractRiskFlag in inputs? False | CustomerSegment? False
Engineered features the model uses: ['ServiceCount', 'Cluster', 'TenureBand',
                                     'ARPU', 'ChargeRatio', 'ValueTier', 'AutoPayFlag']

Missing model inputs after serving-path build: NONE
BATCH SCORING OK - scored 200 customers
probability range: 0.0251 to 0.7443 | mean 0.2656
```

Listing 18: Serving-path verification. The mean predicted probability of 0.2656 against an observed base rate of 0.2654 is independent evidence that calibration is sound.

11. Phase 1 — Data Collection and Understanding

11.1 Purpose

Phase 1 performs no transformation. It establishes ground truth about the raw file before any cleaning decision is taken: how large the dataset is, what type each column holds, where values are missing, and what the baseline churn rate is. It writes nothing to disk. This phase was not modified during remediation.

Property	Value
Input	<code>data/Customer-Churn.csv</code>
Output	console report only
Lines of code	97
Status	Unchanged by remediation

Table 15: Phase 1 interface summary.

11.2 Code walk-through

The TotalCharges quality check

```
total_charges_numeric = pd.to_numeric(df["TotalCharges"], errors="coerce")
invalid_total_charges = total_charges_numeric.isna().sum()

if invalid_total_charges > 0:
    print(df.loc[
        total_charges_numeric.isna(),
        [ID_COLUMN, "tenure", "MonthlyCharges", "TotalCharges"]
    ].head().to_string(index=False))
```

This is the most consequential block in phase 1. TotalCharges is stored as text because eleven rows contain a single space rather than a number, and one non-numeric entry forces the whole column to object dtype. The `errors="coerce"` argument converts those entries to NaN instead of raising, so they can be counted. Crucially, the script then prints the tenure of each affected row — which is what reveals that all eleven are zero-tenure customers, justifying the targeted imputation in phase 2 rather than a blind median fill.

Baseline churn rate

```
churn_rate = (df[TARGET_COLUMN] == "Yes").mean() * 100
print(f"\nBaseline churn rate: {churn_rate:.2f}%")
```

The idiom is worth noting: the comparison produces a boolean Series, and because Python treats True as 1, taking its mean yields the proportion directly.

11.3 Results

Diagnostic	Measured result	Consequence
Dataset shape	7 043 × 21	no sampling needed
Explicit null values	0	no general imputation required
Object-typed columns	18 of 21	one-hot encoding needed
TotalCharges dtype	object (should be numeric)	numeric cast required in phase 2
Blank TotalCharges	11 rows, all with tenure = 0	targeted fill with 0.0, not median

Churn distribution	5 174 No / 1 869 Yes	class weighting required
Baseline churn rate	26.54 %	accuracy floor is 73.46 %

Table 16: Phase 1 profiling results.

12. Phase 2 — Data Cleaning

12.1 Purpose

Phase 2 converts the raw file into a validated analytical dataset: it repairs TotalCharges, resolves missing values with domain reasoning, removes duplicates, validates numeric ranges, caps outliers and standardises redundant categorical labels. Unchanged by remediation.

Property	Value
Input	<code>data/Customer-Churn.csv</code>
Output	<code>data/Customer-Churn-Cleaned.csv</code> — 7 043 × 21
Rows removed	0
Status	Unchanged by remediation

Table 17: Phase 2 interface summary.

12.2 Code walk-through

Whitespace normalisation

```
df = df.replace(r"^\s*$", pd.NA, regex=True)
```

A single regular expression applied across the frame. It matters because pandas does not treat a space character as missing: without this line `isna()` reports zero missing values while eleven cells are in fact empty, and every subsequent missing-value check returns a false clean bill of health.

Domain-driven imputation

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# For newly joined customers with tenure 0, TotalCharges should be 0
df.loc[(df["tenure"] == 0) & (df["TotalCharges"].isna()), "TotalCharges"] = 0.0

remaining_missing = df["TotalCharges"].isna().sum()
if remaining_missing > 0:
    df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
```

This is the most defensible decision in the cleaning stage. The naive treatment is a median fill, which here would assign about 1 395 units of accumulated revenue to customers who joined this month and have been billed nothing — a fabrication that would distort the ARPU and ChargeRatio features and corrupt the revenue reasoning the whole project rests on. Because phase 1 established that all eleven blanks belong to zero-tenure customers, the correct value is known to be zero. The median branch remains as a defensive fallback and never executes against this data.

Outlier capping by interquartile range

```
for column in ["tenure", "MonthlyCharges", "TotalCharges"]:
    q1, q3 = df[column].quantile(0.25), df[column].quantile(0.75)
```

```
iqr = q3 - q1
lower_bound, upper_bound = q1 - (1.5 * iqr), q3 + (1.5 * iqr)
df[column] = df[column].clip(lower=lower_bound, upper=upper_bound)
```

Tukey's rule, applied as capping rather than deletion so the record and its label are preserved. Executed against this dataset it caps nothing at all — all three columns are bounded by their own business rules and the measured outlier count is zero in every case. The step is protective rather than corrective.

12.3 Results

Operation	Records affected	Outcome
Whitespace → null conversion	11 cells	blanks made detectable
Numeric cast of TotalCharges	7 043 rows	object → float64
Zero-tenure imputation	11 rows	set to 0.0
Median fallback fill	0 rows	branch not reached
Duplicate removal	0 rows	dataset already unique
Range validation	0 violations	all values plausible
IQR capping	0 values	no extreme values present
Label standardisation	7 columns	3 levels reduced to 2

Table 18: Every cleaning operation with its measured effect.

Column	Min	Max	Mean	Median	Std dev
tenure (months)	0.00	72.00	32.37	29.00	24.56
MonthlyCharges	18.25	118.75	64.76	70.35	30.09
TotalCharges	0.00	8 684.80	2 279.73	1 394.55	2 266.79

Table 19: Descriptive statistics after cleaning. All 7 043 records retained.

13. Phase 3 — Exploratory Data Analysis

13.1 Purpose

Phase 3 answers the question that determines whether the modelling effort is worthwhile: does churn concentrate in identifiable groups? It works through univariate distributions, bivariate churn-rate profiling, correlation structure and tenure cohorts, writing nine charts. Unchanged by remediation.

13.2 The reusable churn-rate routine

```
def create_churn_rate_chart(column, title, filename, order=None):
    summary = (df.groupby(column, observed=False)["ChurnFlag"]
               .agg(["mean", "count"]).reset_index())
    summary["ChurnRate"] = (summary["mean"] * 100).round(2)

    ax = sns.barplot(data=summary, x=column, y="ChurnRate",
                     order=order, color="teal")
    for container in ax.containers:
        ax.bar_label(container, fmt="%0.1f%%", padding=3)
```

Five of the nine charts come from this one function. It aggregates both the mean and the count in a single pass — the count matters, because a 60 % churn rate over twelve customers is noise while the same rate over two thousand is a finding — and annotates each bar with its exact value. Mapping churn to a 0/1 integer is what makes a churn rate an arithmetic mean, so any `groupby(...).mean()` returns a rate directly.

13.3 Results: univariate distributions

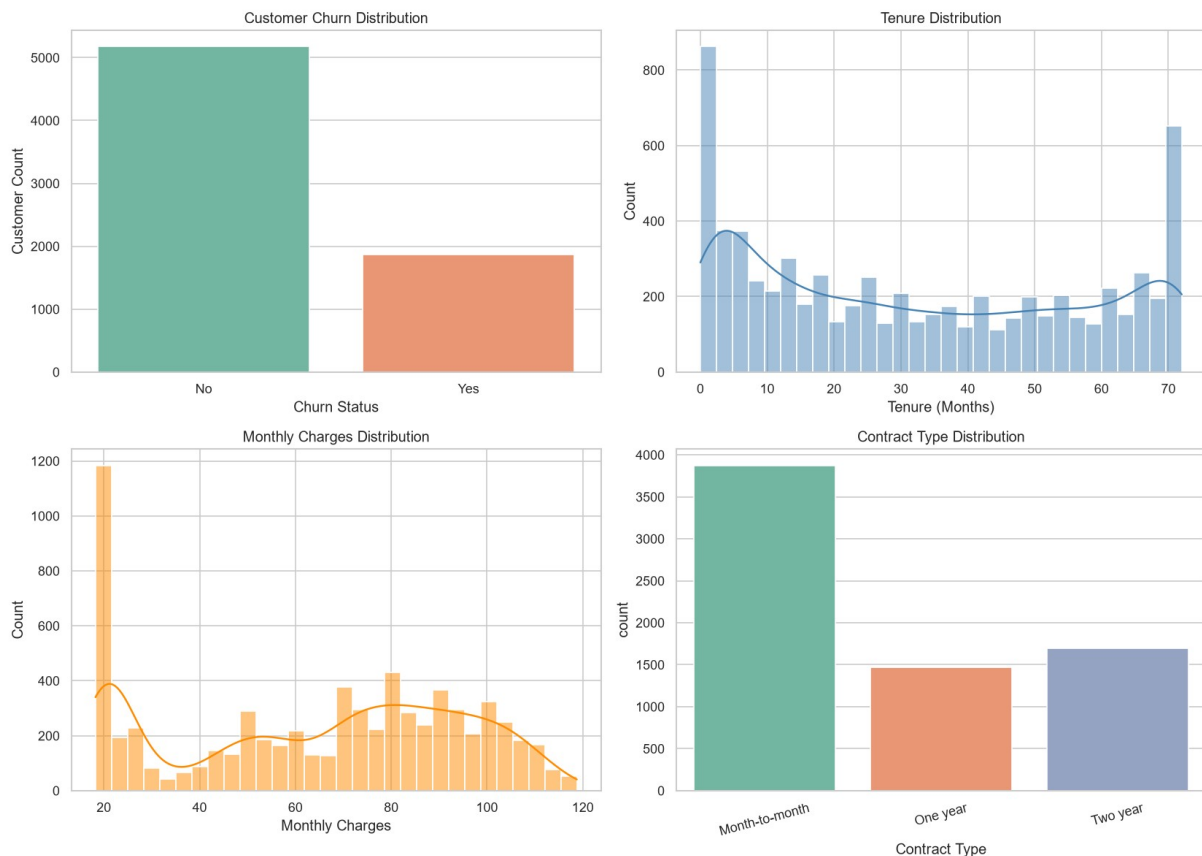


Figure 4: Univariate distributions: churn balance, tenure, monthly charges and contract type.

Three shapes drive the rest of the analysis. Tenure is strongly bimodal — a large mass of very new customers and a second peak at the 72-month ceiling. Monthly charges are bimodal too, separating a low-cost phone-only population near 20 from a bundled population above 70. And month-to-month is by a wide margin the most common contract, which matters because it is also by far the riskiest.

13.4 Results: churn rate by attribute

Attribute	Level	Customers	Churn rate	Lift vs 26.54 % base
Contract	Month-to-month	3 875	42.71 %	+16.17 pp
	One year	1 473	11.27 %	-15.27 pp
	Two year	1 695	2.83 %	-23.71 pp
Payment method	Electronic check	2 365	45.29 %	+18.75 pp
	Mailed check	1 612	19.11 %	-7.43 pp
	Bank transfer (automatic)	1 544	16.71 %	-9.83 pp
	Credit card (automatic)	1 522	15.24 %	-11.30 pp
Internet service	Fiber optic	3 096	41.89 %	+15.35 pp
	DSL	2 421	18.96 %	-7.58 pp
	No internet	1 526	7.40 %	-19.14 pp
Tech support	No	4 999	31.19 %	+4.65 pp
	Yes	2 044	15.17 %	-11.37 pp
Online security	No	5 024	31.33 %	+4.79 pp
	Yes	2 019	14.61 %	-11.93 pp
Senior citizen	Yes	1 142	41.68 %	+15.14 pp
	No	5 901	23.61 %	-2.93 pp
Paperless billing	Yes	4 171	33.57 %	+7.03 pp
	No	2 872	16.33 %	-10.21 pp
Dependents	No	4 933	31.28 %	+4.74 pp
	Yes	2 110	15.45 %	-11.09 pp
Partner	No	3 641	32.96 %	+6.42 pp
	Yes	3 402	19.66 %	-6.88 pp
Gender	Female	3 488	26.92 %	+0.38 pp
	Male	3 555	26.16 %	-0.38 pp

Table 20: Churn rate by attribute level. Lift is stated in percentage points against the 26.54 % base rate.

The gender row is included deliberately, because a null result is as informative as a positive one. A 0.76-point difference across 7 043 records is indistinguishable from noise, which tells us churn here is driven by commercial structure — contract, payment mechanism, product mix — and not by demographics. Retention budget spent on demographic targeting would be wasted.

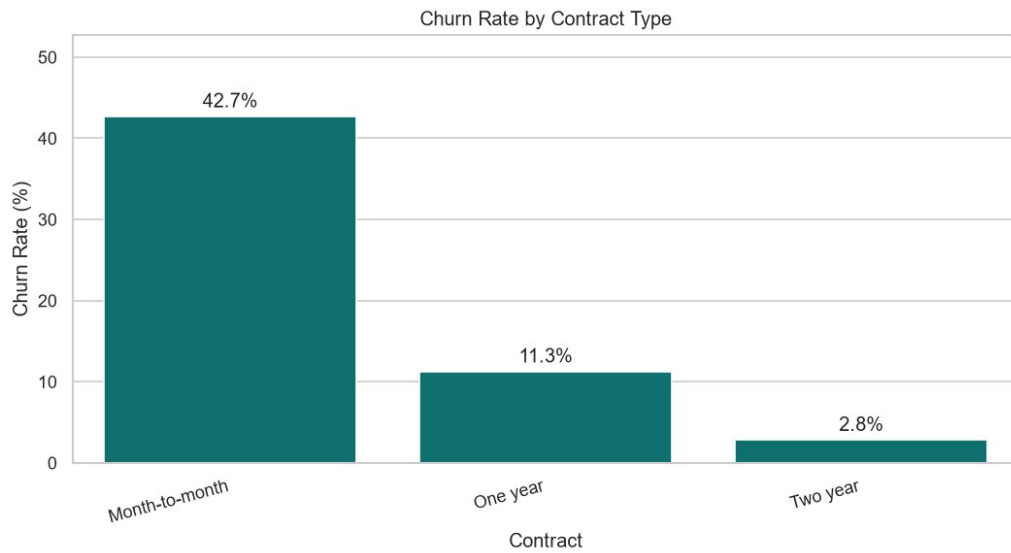


Figure 5: Churn rate by contract type. A two-year contract reduces churn to roughly one-fifteenth of the month-to-month rate.

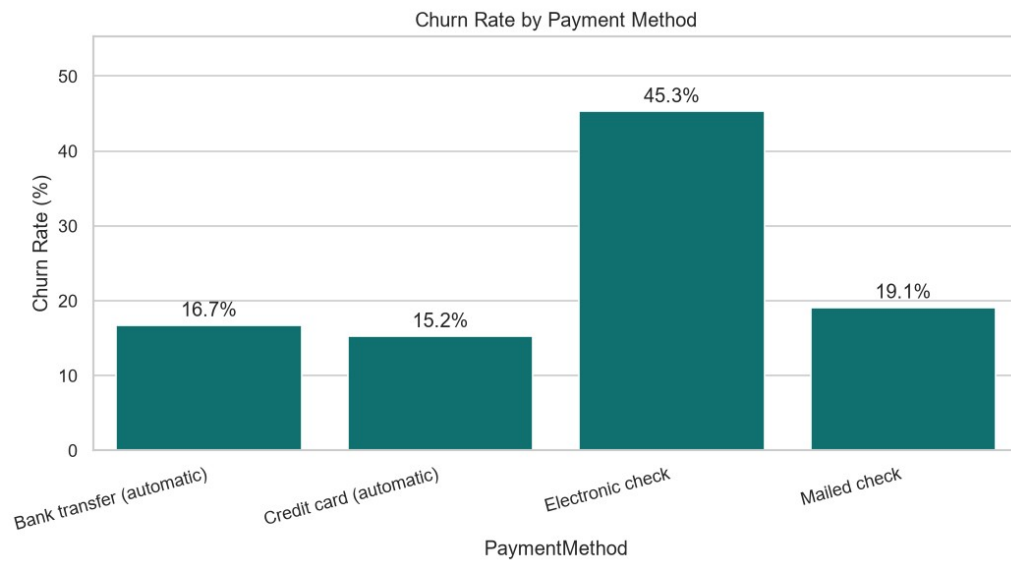


Figure 6: Churn rate by payment method. Electronic check is the highest-risk level of any attribute examined.

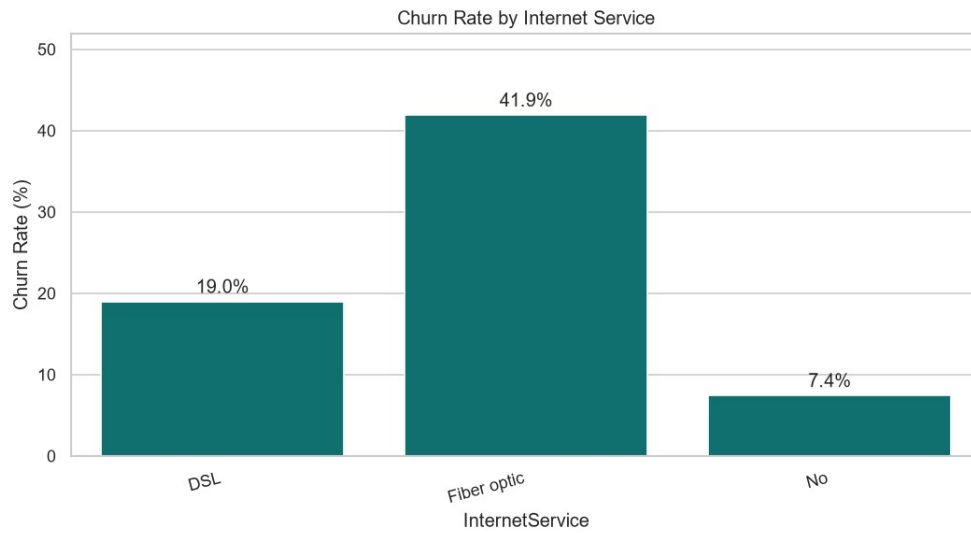


Figure 7: Churn rate by internet service.

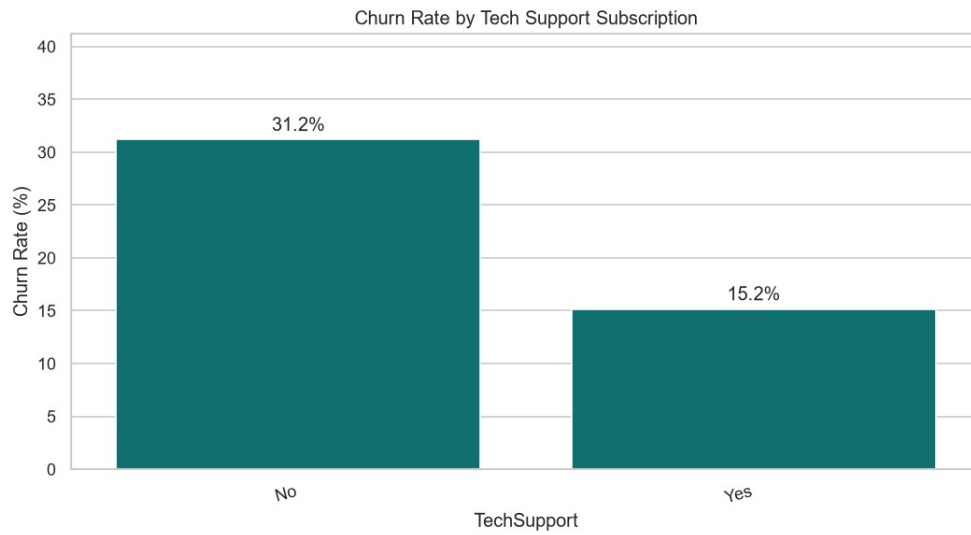


Figure 8: Churn rate by tech-support subscription.

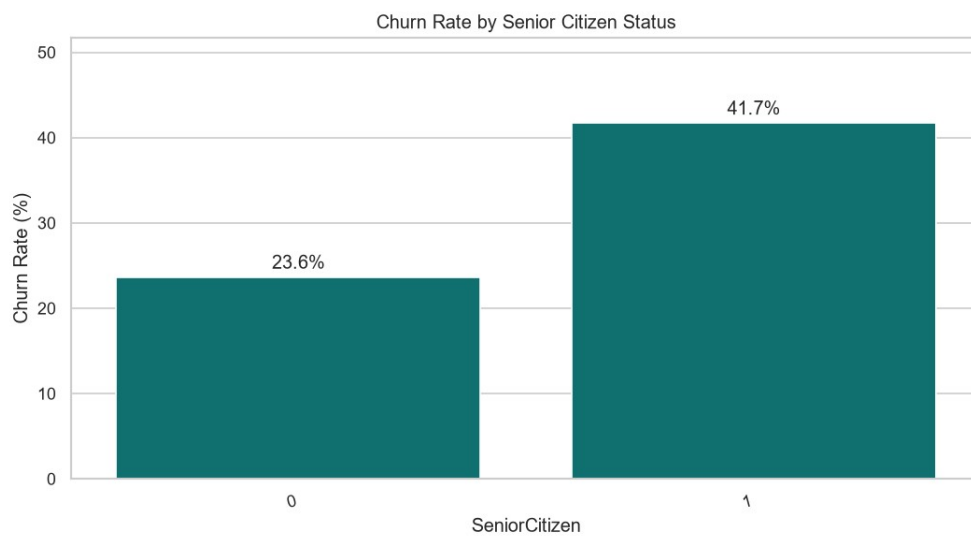


Figure 9: Churn rate by senior-citizen status.

13.5 Results: correlation structure

Variable pair	Pearson r	Reading
tenure ↔ ChurnFlag	-0.352	the strongest single linear signal
TotalCharges ↔ ChurnFlag	-0.198	a proxy for tenure, not an independent driver
MonthlyCharges ↔ ChurnFlag	+0.193	higher recurring price raises risk
SeniorCitizen ↔ ChurnFlag	+0.151	consistent with the 41.68 % rate above
tenure ↔ TotalCharges	+0.826	near-collinear by construction
MonthlyCharges ↔ TotalCharges	+0.651	total is rate multiplied by duration

Table 21: Pearson correlations among the numeric attributes and the churn flag.

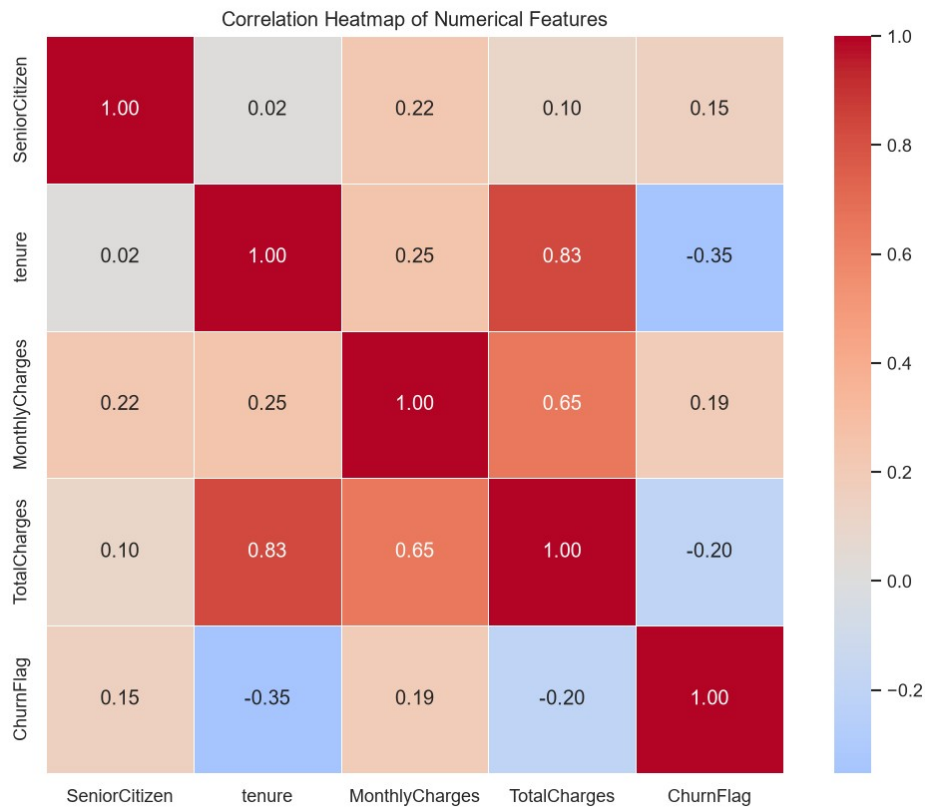


Figure 10: Correlation heat map of the numeric attributes.

The 0.826 correlation between tenure and TotalCharges has the most practical consequence. The two are near-collinear because total charges are, almost by definition, monthly charges multiplied by tenure. For tree-based models this is harmless. For Logistic Regression it inflates the standard errors of both coefficients; the phase 6 implementation mitigates it with an L2 penalty, which shrinks correlated coefficients toward one another rather than letting one dominate arbitrarily.

13.6 Results: behavioural and cohort analysis

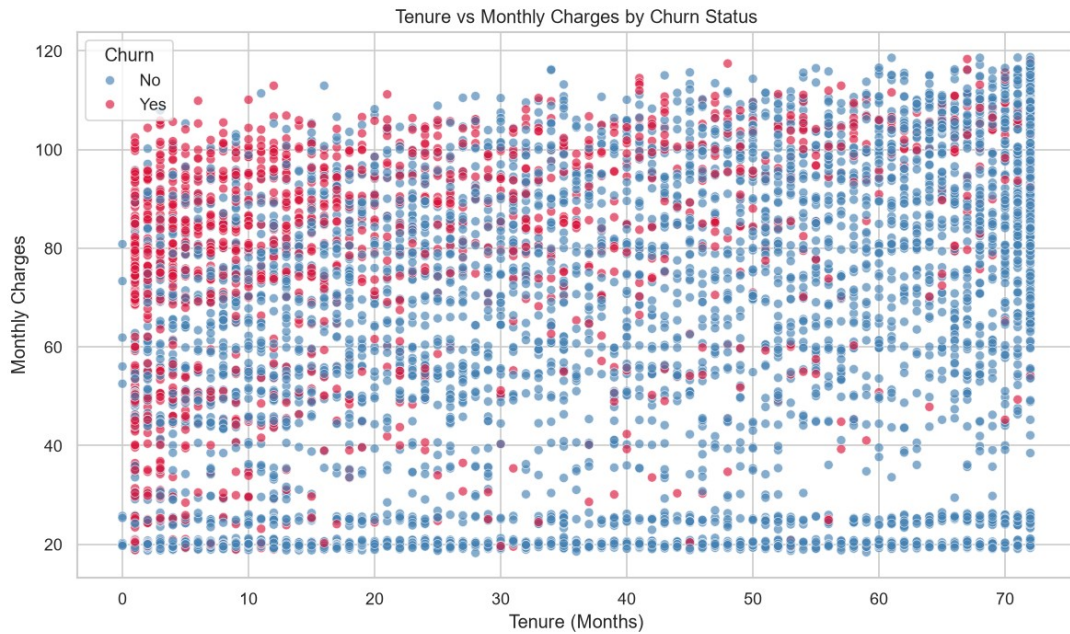


Figure 11: Tenure against monthly charges, coloured by churn outcome.

This is the most operationally useful chart phase 3 produces. Churned customers concentrate visibly in the upper-left region — short tenure combined with high monthly charges. That is exactly the profile of a customer who recently signed up for an expensive bundle and has not yet accumulated any reason to stay. It is also, as Section 15 shows, the profile K-Means isolates without being told anything about churn.

Tenure band	Customers	Share	Churn rate	Reading
0-6 months	1 481	21.03 %	52.94 %	more than half leave within two quarters
7-12 months	705	10.01 %	35.89 %	still above base
13-24 months	1 024	14.54 %	28.71 %	approaching base
25-48 months	1 594	22.63 %	20.39 %	settled
49-72 months	2 239	31.79 %	9.51 %	loyal core

Table 22: Churn rate by tenure cohort, showing monotonic decay of risk with relationship age.

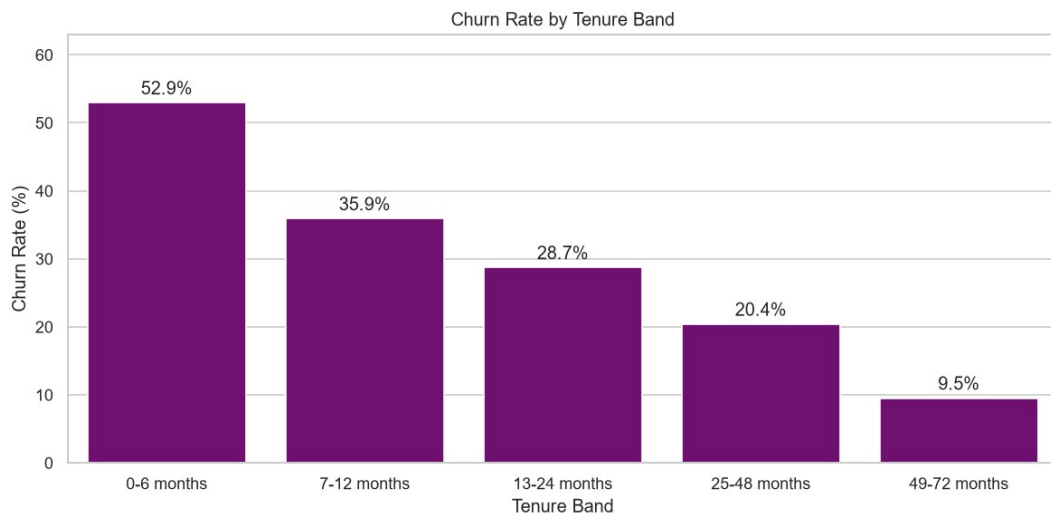


Figure 12: Churn rate by tenure band. Risk decays monotonically from 52.94 % to 9.51 %.

The cohort table carries the clearest strategic message in the exploratory phase. A customer in their first six months is five and a half times more likely to leave than one past four years. Retention effort therefore has a window: intervening at month two addresses a 52.94 % risk, while the same intervention at month fifty addresses a 9.51 % risk and is largely wasted. Onboarding, not long-term loyalty programming, is where retention spend earns its return.

14. Phase 4 — Feature Engineering

14.1 Purpose

Phase 4 translates the exploratory findings into model inputs. Each of the seven constructed features encodes a specific business hypothesis found in phase 3, rather than an arbitrary combination of columns. **Modified during remediation** to build features through the shared module, persist the tier boundaries and export an unscaled copy.

Property	Value
Input	<code>data/Customer-Churn-Cleaned.csv</code>
Outputs	<code>Customer-Churn-Features-Unscaled.csv</code> (7 043 × 8), <code>Customer-Churn-Model-Ready.csv</code> (7 043 × 36), <code>feature_engineering_artifacts.joblib</code>
Features constructed	7
Status	Modified — defect 3

Table 23: Phase 4 interface summary.

14.2 The seven engineered features

Feature	Construction	Business hypothesis
<code>TenureBand</code>	<code>pd.cut</code> on tenure into 5 unequal bands	risk decays non-linearly with relationship age; a linear term cannot express the 52.94 % → 9.51 % curve
<code>ARPU</code>	<code>TotalCharges / tenure</code> , guarded at tenure = 0	revenue actually realised per active month, as distinct from list price
<code>ChargeRatio</code>	<code>MonthlyCharges / ARPU</code>	above 1 means the customer now pays more than their historical average — a recent price rise
<code>ServiceCount</code>	count of "Yes" across 6 add-on columns	product entanglement; each service raises the switching cost
<code>ContractRiskFlag</code>	1 if contract is month-to-month	a readable flag for the strongest driver in phase 3; kept in the data but excluded from the model because it duplicates <code>Contract</code> (Section 22.2)
<code>ValueTier</code>	<code>pd.qcut</code> on ARPU into 4 quartiles	revenue-based prioritisation, so effort can be ranked by worth
<code>AutoPayFlag</code>	1 if payment method is automatic	automatic payment removes a recurring cancellation decision point

Table 24: The seven engineered features and the hypothesis each encodes.

14.3 Code walk-through

Division guarded against zero tenure

```
def add_arpu(df):
    df["ARPU"] = 0.0
    non_zero_tenure = df["tenure"] > 0
    df.loc[non_zero_tenure, "ARPU"] = (
        df.loc[non_zero_tenure, "TotalCharges"] / df.loc[non_zero_tenure, "tenure"]
    )
    return df
```

Both derived ratios divide by a quantity that is zero for the eleven newly joined customers. The pattern used — initialise the column to a safe default, compute a boolean mask of rows where the division is defined, assign only into those rows — avoids producing infinity entirely. Computing first and repairing afterwards is

fragile, because an infinity that survives into a scaler poisons the mean and standard deviation of the whole column and silently corrupts every other row.

Tier boundaries are now fitted and persisted

```
def fit_value_tier_edges(df):
    """Learn ARPU quartile edges on the training data and return them."""
    _, edges = pd.qcut(df["ARPU"], q=4, retbins=True, duplicates="drop")
    edges = list(edges)
    edges[0], edges[-1] = -np.inf, np.inf      # so unseen values still bin
    return edges
```

Listing 19: Added during remediation. Without persisted edges the dashboard would recompute quartiles from whatever batch it was given, so the same customer could land in different tiers in different uploads.

14.4 Results

Property	Before phase 4	After phase 4
Rows	7 043	7 043
Columns (model-ready copy)	21	36
Unscaled feature export	—	7 043 × 8 (new)
Data types	18 object, 3 numeric	all numeric
Missing values	0	0

Table 25: Structural transformation applied by phase 4.

Service count	Customers	Share	Churn rate	Reading
0 add-ons	2 219	31.51 %	21.41 %	largely no-internet customers, structurally low risk
1 add-on	966	13.72 %	45.76 %	the peak-risk group in the entire base
2 add-ons	1 033	14.67 %	35.82 %	risk falling
3 add-ons	1 118	15.87 %	27.37 %	at the base rate
4 add-ons	852	12.10 %	22.30 %	below base
5 add-ons	571	8.11 %	12.43 %	entangled
6 add-ons	284	4.03 %	5.28 %	lowest risk of any group

Table 26: Churn rate by number of add-on services, showing a non-monotonic relationship.

The ServiceCount profile is the most interesting result phase 4 produces, because it is not monotonic. Risk peaks at exactly one add-on (45.76 %) then falls steadily to 5.28 % at six. The zero-add-on group sits low at 21.41 % for a different reason: it is dominated by customers with no internet at all, who have little to churn from. A single add-on is the worst place for a customer to sit — paying for a bundle without being anchored by it. Because a straight line cannot represent a curve that rises then falls, this is also a concrete argument for tree-based models.

Feature	Level	Customers	Churn rate
ValueTier	Low	1 761	11.07 %
	Medium	1 762	25.37 %
	High	1 759	36.33 %
	Premium	1 761	33.39 %
ContractRiskFlag	0 (annual or two-year)	3 168	6.76 %
	1 (month-to-month)	3 875	42.71 %
AutoPayFlag	0 (manual payment)	3 977	34.67 %

	1 (automatic payment)	3 066	15.98 %
--	-----------------------	-------	---------

Table 27: Churn rate across the engineered flags and tiers, measured on the cleaned dataset.

Churn rises sharply with value tier before easing slightly at the top. The High tier is the commercially dangerous one: it combines a high churn rate with substantial per-customer revenue, so it carries the largest revenue at risk. The slight decline in Premium is consistent with those customers being heavily bundled, which ServiceCount has already shown to be protective.

15. Phase 5 — Customer Segmentation

15.1 Purpose

Phase 5 groups the customer base into behavioural segments without reference to the churn label. The value of unsupervised work here is that the groups are defined by how customers behave rather than by what happened to them, so they remain meaningful for customers whose outcome is not yet known.

Modified during remediation for defects 2 and 3.

Property	Value
Inputs	Customer-Churn-Cleaned.csv + Customer-Churn-Features-Unscaled.csv
Outputs	Customer-Churn-Segmented.csv (7 043 × 31), customer_segmentation_model.joblib, 4 charts
Algorithm	KMeans, n_init=10, random_state=42, OPTIMAL_K = 4
Clustering dimensions	tenure, ServiceCount, MonthlyCharges
Status	Modified — defects 2 and 3

Table 28: Phase 5 interface summary.

15.2 Choice of clustering dimensions

The three dimensions are an RFM adaptation to a subscription business, where the classical recency-frequency-monetary triple does not translate directly:

Dimension	RFM analogue	Why it is the right proxy
tenure	Recency / longevity	every active subscriber transacted this month, so recency is uninformative; relationship length is the meaningful time dimension
ServiceCount	Frequency / engagement	purchase frequency is fixed at one per month by the billing cycle; breadth of product adoption is the true engagement signal
MonthlyCharges	Monetary	the recurring rate is forward-looking, whereas total charges merely restate tenure

Table 29: Mapping of classical RFM dimensions onto subscription-business proxies.

15.3 Standardisation before clustering

```
X = df[["tenure", "ServiceCount", "MonthlyCharges"]].copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Standardisation is a correctness requirement here, not a convenience. K-Means assigns points by Euclidean distance, and the raw dimensions have incompatible scales: tenure spans 0 to 72, monthly charges 18.25 to 118.75, service count 0 to 6. Unscaled, monthly charges would contribute roughly four hundred times more squared distance than service count, and the clusters would be little more than a partition of the price axis.

15.4 Selecting the number of segments

K	Inertia	Inertia reduction	Silhouette	Assessment
2	10 782.45	—	0.4163	highest silhouette, but only splits cheap from expensive
3	7 295.42	32.3 %	0.3968	improving, still coarse
4	5 615.08	23.0 %	0.4042	selected — elbow region, silhouette recovers
5	4 647.43	17.2 %	0.3641	silhouette degrades

6	3 812.39	18.0 %	0.3704	no separation gain
---	----------	--------	--------	--------------------

Table 30: Elbow and silhouette results for K from 2 to 6.

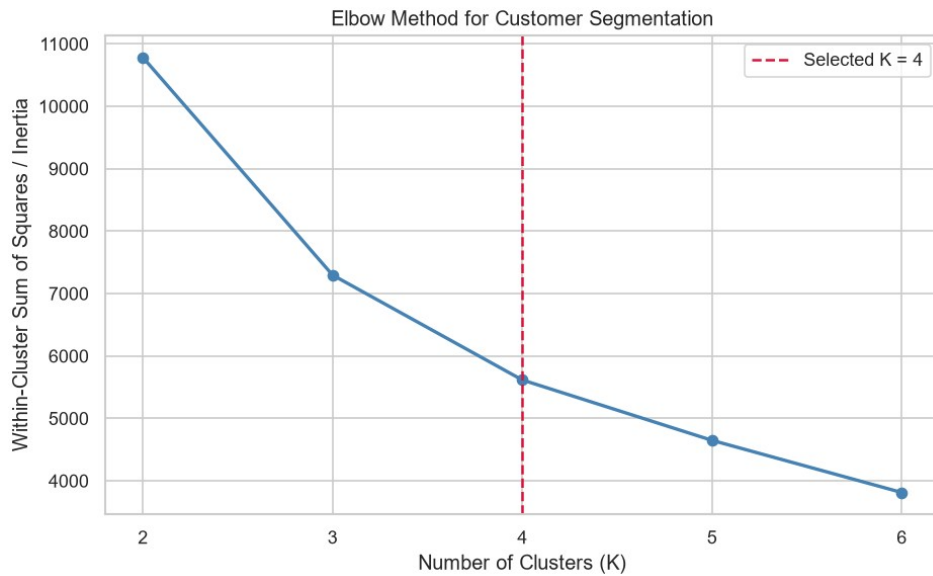


Figure 13: Elbow analysis. The inertia curve flattens noticeably after K = 4.

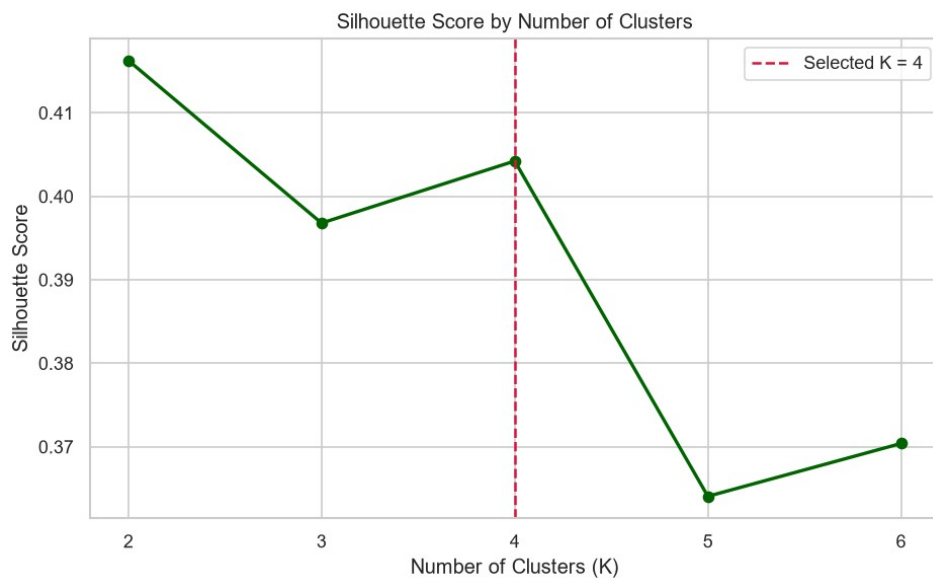


Figure 14: Silhouette score by K.

The two criteria disagree, and the resolution deserves stating openly. K = 2 attains the highest silhouette at 0.4163, so a purely geometric reading would choose two clusters. Four is selected on three grounds: the inertia curve shows its clearest elbow there (23.0 % reduction from K = 3 against only 17.2 % moving to K = 5); the silhouette at K = 4 is within 0.012 of the K = 2 value, so almost no cluster quality is sacrificed; and decisively, a two-cluster solution divides the base into cheap and expensive customers, supporting no retention decision that pricing data alone would not already support.

15.5 Segment profiles after centroid-based naming

Segment	Cluster	Customers	Share	Mean tenure	Mean monthly	Mean services	Churn rate
New High-Spend	3	2 373	33.69 %	15.05	77.58	1.83	47.37 %
New Budget	0	1 521	21.60 %	10.16	29.58	0.30	24.19 %
Established High-Value	1	2 158	30.64 %	56.89	90.59	4.16	15.43 %
Established Budget	2	991	14.07 %	54.54	31.81	0.58	4.44 %

Table 31: The four segments, ordered by churn rate. Names derive from centroid geometry; churn rate is a measured outcome, not a naming input.

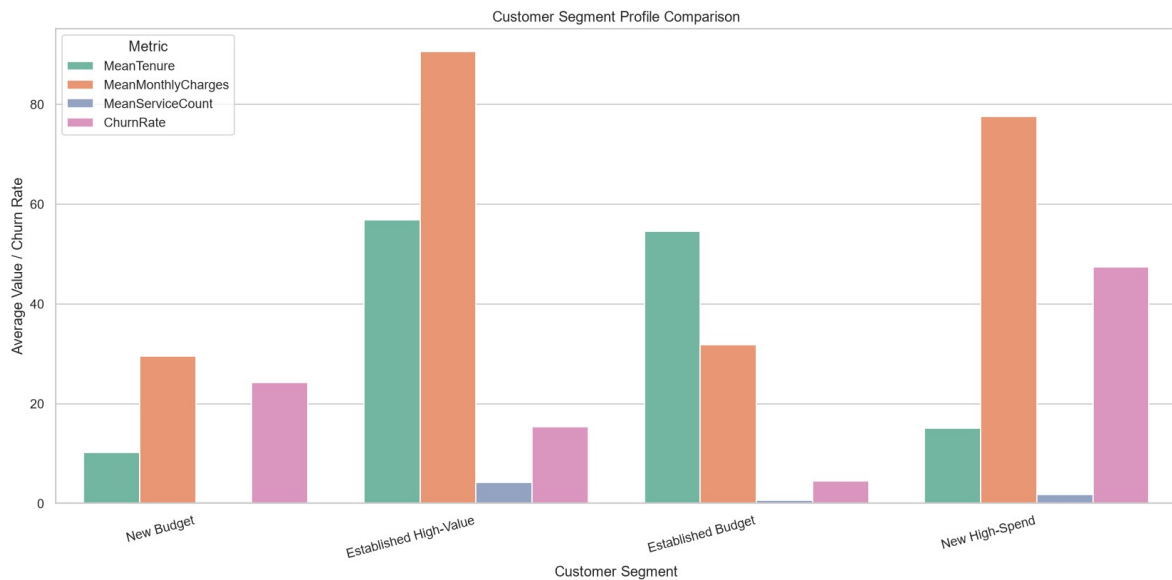


Figure 15: Segment profile comparison across tenure, monthly charges, service count and churn rate.

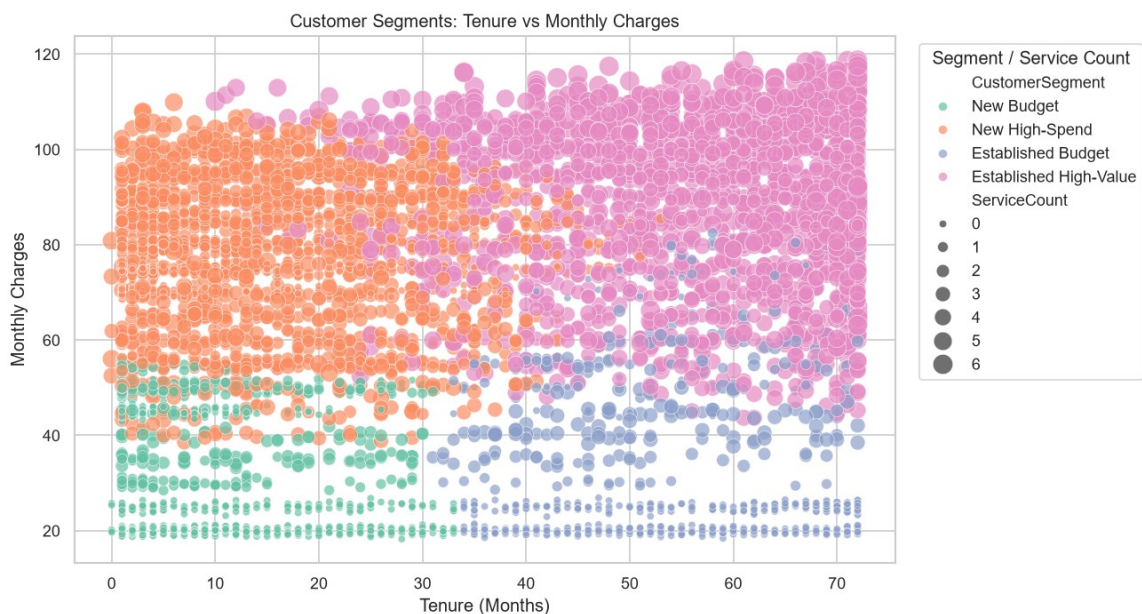


Figure 16: Segments plotted on tenure against monthly charges, with marker size encoding service count.

Reading the four segments

- **New High-Spend — the retention priority.** 2 373 customers, a third of the base, averaging fifteen months of tenure and 77.58 in monthly charges. Nearly half leave. This segment combines the highest churn rate with substantial recurring revenue, making it the largest pool of recoverable revenue in the business.

- **Established High-Value — protect, do not discount.** 2 158 customers averaging fifty-seven months, 90.59 per month and 4.16 add-on services. At 15.43 % their churn is well below base, and their 5 160 mean accumulated revenue makes each loss expensive. The correct posture is service quality and early-warning monitoring.
- **New Budget — low value, moderate risk.** 1 521 customers on 29.58 per month with almost no add-on adoption. At 24.19 % their churn sits below base and their low revenue means each loss is small. Retention spend here earns a poor return.
- **Established Budget — leave alone.** 991 long-tenured customers on minimal spend, churning at just 4.44 %. The most stable group in the base; no intervention needed.

The segments separate churn strongly without ever seeing it.

Churn rates across the four segments span 4.44 % to 47.37 %, a ten-fold spread, and K-Means was given only tenure, service count and monthly charges. This is the evidence that the behavioural structure is real rather than an artefact of the naming rule — and it is exactly the demonstration the leaking version could not make, because there the separation was partly built into the labels by construction.

16. Phase 6 — Model Development

16.1 Purpose

Phase 6 trains five classifiers, selects among them by cross-validated ROC-AUC, tunes the strongest family by randomized search, evaluates every candidate on the hold-out partition and serialises the winner with its full preprocessing pipeline. **Modified during remediation** for defects 1 and 2, and again in the second round so labels are assigned by an explicit probability threshold.

Property	Value
Input	<code>data/Customer-Churn-Segmented.csv</code> (7 043 × 31)
Outputs	<code>best_churn_model.joblib</code> , <code>phase6_model_performance.csv</code> , 3 charts
Models trained	5 (LR, RF, SVM, XGBoost baseline, XGBoost tuned)
Split	<code>TEST_SIZE = 0.20</code> — 5 634 train / 1 409 test
Labelling	<code>predict_proba >= DECISION_THRESHOLD</code> (0.50)
Status	Modified — defects 1, 2 and issue 4

Table 32: Phase 6 interface summary.

16.2 Leakage-safe preprocessing

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))])

preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_transformer, numeric_features),
    ("categorical", categorical_transformer, categorical_features)])
```

This is the most important structural decision in phase 6, and the delivered code got it right. Wrapping imputation, scaling and encoding inside a `ColumnTransformer` that is itself the first pipeline step means the fitted statistics — medians, means, standard deviations, category vocabularies — are learned only from the training rows of each fold and then applied to the validation rows. It is also why defect 3 had to be fixed by merging **unscaled** features: handing this pipeline pre-scaled columns would have defeated the very protection it provides.

16.3 Class-imbalance correction

```
negative_class_count = (y_train == 0).sum()
positive_class_count = (y_train == 1).sum()
scale_pos_weight = negative_class_count / positive_class_count
```

With a 73.46 / 26.54 split, an unweighted learner minimises its loss most easily by predicting the majority class. Three of the four baseline models counter this with `class_weight="balanced"`; XGBoost exposes the same correction through `scale_pos_weight`, approximately 2.77 on the training partition. This is why every weighted model achieves recall above 0.78.

16.4 Hyper-parameter tuning

```
xgb_search = RandomizedSearchCV(
    estimator=xgb_pipeline,
    param_distributions=xgb_parameter_grid,    # 10 parameters
    n_iter=30, scoring="roc_auc", cv=cv,
    random_state=RANDOM_STATE, n_jobs=-1, refit=True)
```

The grid spans ten hyper-parameters with four candidate values for most, defining a space of well over a million combinations. Exhaustive search is infeasible; thirty random draws across five folds is 150 fits, a budget that completes in minutes while reliably locating a near-optimal region. `refit=True` retrains the best configuration on the entire training partition once the search concludes.

16.5 Cross-validation results

Model	Mean CV ROC-AUC	Std dev	Reading
Logistic Regression	0.8465	0.0113	strongest on cross-validation
Random Forest	0.8455	0.0093	within one standard deviation
Support Vector Machine	0.8298	0.0109	clearly behind
XGBoost Baseline	0.8003	0.0047	untuned; 2 000 rounds at lr 0.10 overfits

Table 33: Five-fold stratified cross-validation on the training partition (26-column matrix).

16.6 Labelling predictions consistently

```
DECISION_THRESHOLD = 0.50

predicted_probabilities = pipeline.predict_proba(X_test)[: , 1]
predicted_labels = (predicted_probabilities >= DECISION_THRESHOLD).astype(int)
```

Listing 20a: Phase 6 labels predictions exactly as phase 7 does (second-round correction, Section 22.1).

Phase 6 previously called `pipeline.predict()`. For Random Forest and XGBoost that is the same as thresholding the probability at 0.50, but for an `SVC` it is not: `predict()` uses the sign of the decision function while `predict_proba` uses Platt scaling fitted by internal cross-validation. Using one explicit threshold everywhere makes the operating point visible and makes phases 6 and 7 agree.

16.7 Hold-out test results

Model	ROC-AUC	Accuracy	Precision	Recall	F1	Rank
Tuned XGBoost	0.8472	0.7339	0.4992	0.8262	0.6224	1
Logistic Regression	0.8448	0.7459	0.5139	0.7914	0.6232	2
Random Forest	0.8425	0.7587	0.5309	0.7807	0.6320	3
Support Vector Machine	0.8334	0.7850	0.5927	0.6070	0.5997	4
XGBoost Baseline	0.8025	0.7658	0.5608	0.5428	0.5516	5

Table 34: Hold-out performance on the 1 409-record test set, ranked by ROC-AUC. Reproduced from `data/phase6_model_performance.csv`.

Phases 6 and 7 now report identical figures.

Every model evaluated in both phases — Tuned XGBoost, Logistic Regression, Random Forest and the SVM — now shows the same accuracy, precision, recall and F1 in the table above and in the phase 7 table in Section 17.2. Before the second-round correction the SVM appeared at 0.7601 accuracy and 0.7834 recall here, and at 0.7835 and 0.6150 in phase 7.

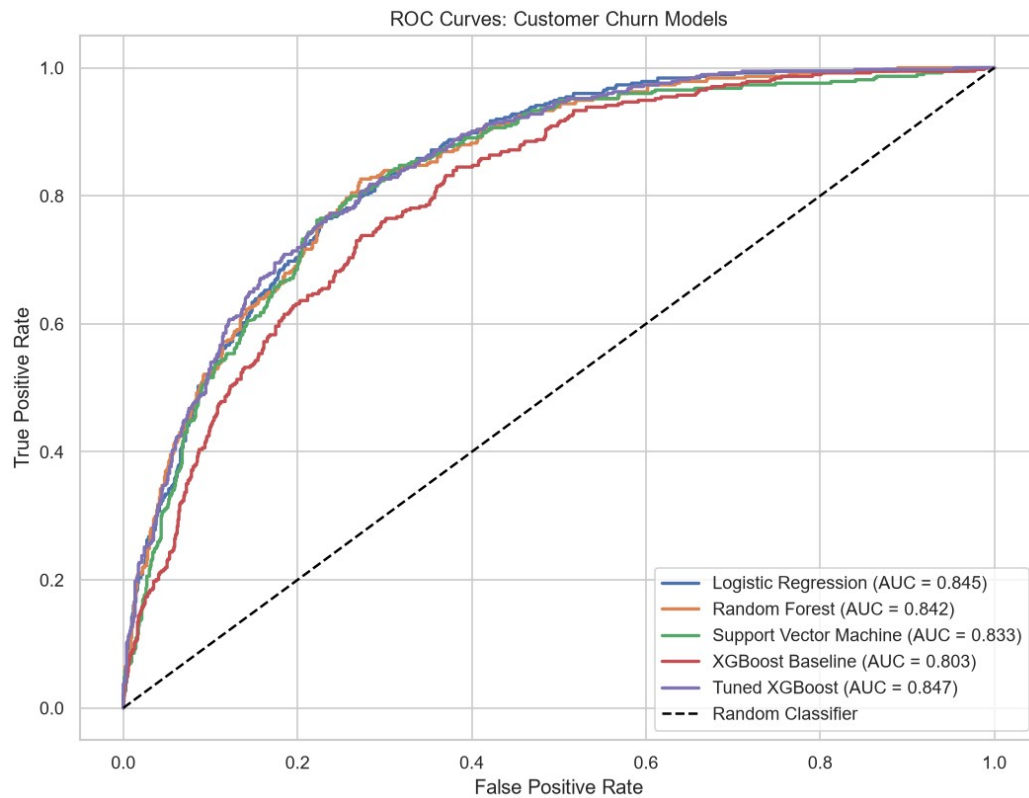


Figure 17: ROC curves for all five models on the hold-out partition.

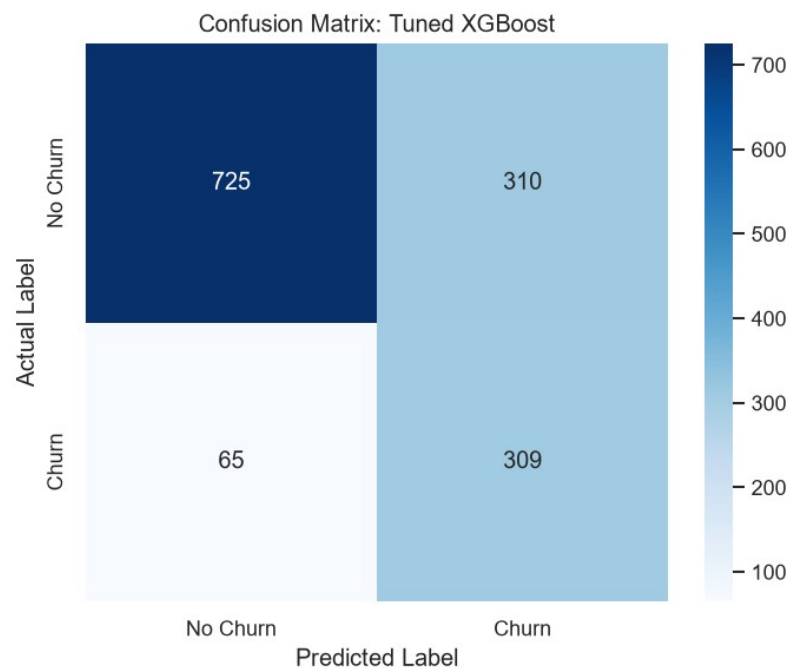


Figure 18: Confusion matrix for the selected model on the 1 409-record hold-out.

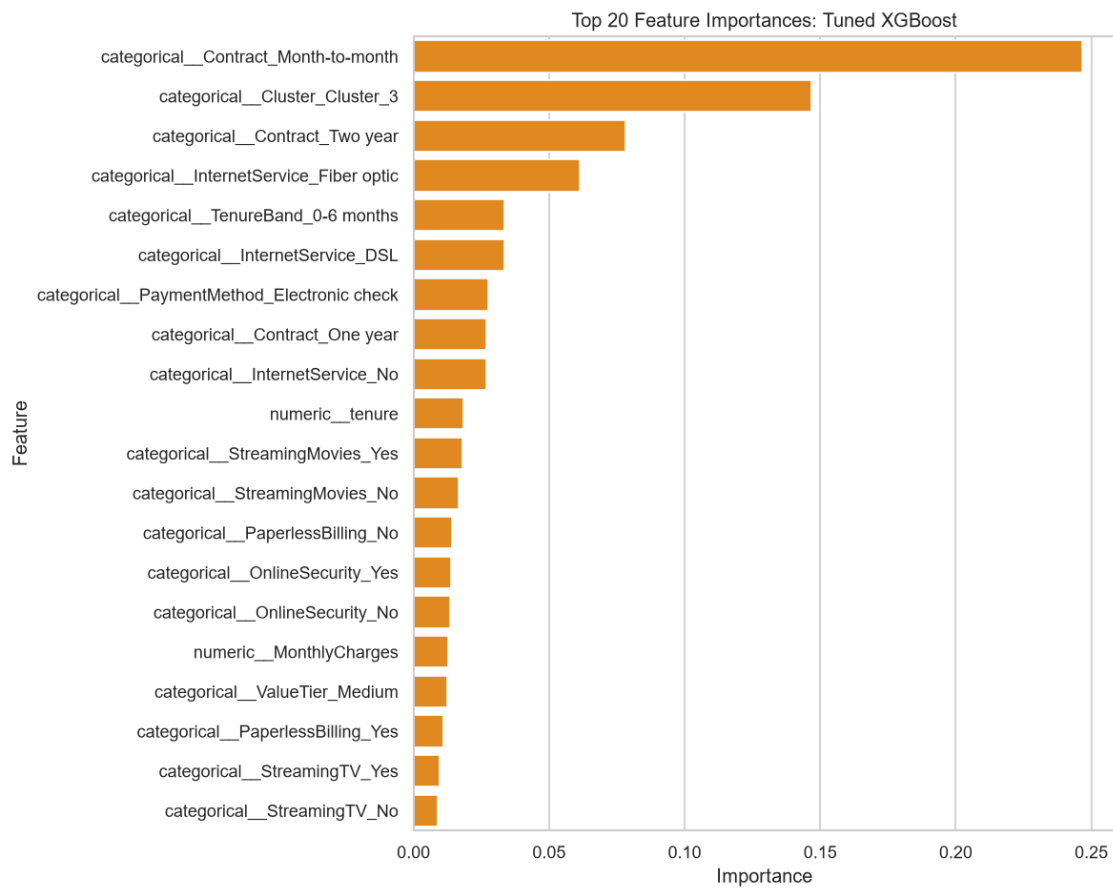


Figure 19: Top twenty features by gain importance in the tuned XGBoost model.

17. Phase 7 — Model Evaluation and Threshold Analysis

17.1 Purpose

Phase 7 re-fits four models on the same split and evaluates them more thoroughly than phase 6: precision-recall curves, full confusion matrices, PR-AUC, and a sweep of the decision threshold from 0.10 to 0.90. **Modified during remediation** for defects 1 and 2.

17.2 Evaluation results with confusion matrices

Model	ROC-AUC	PR-AUC	Acc.	Prec.	Recall	F1	TN	FP	FN	TP
Tuned XGBoost	0.8472	0.6632	0.7339	0.4992	0.8262	0.6224	725	310	65	309
Logistic Regression	0.8448	0.6492	0.7459	0.5139	0.7914	0.6232	755	280	78	296
Random Forest	0.8425	0.6510	0.7587	0.5309	0.7807	0.6320	777	258	82	292
Support Vector Machine	0.8334	0.6269	0.7850	0.5927	0.6070	0.5997	879	156	147	227

Table 35: Phase 7 hold-out evaluation with confusion-matrix cells. Labels are assigned by thresholding predicted probability at 0.50, identically to phase 6.

The confusion matrix for the selected model reads as follows. Of 374 customers who actually churned, the model identified 309 and missed 65. It also raised 310 false alarms. Given that a missed churner costs the customer's remaining lifetime value while a false alarm costs one retention offer, this asymmetry is economically favourable — the business spends 310 unnecessary offers to reach 309 genuine churners.

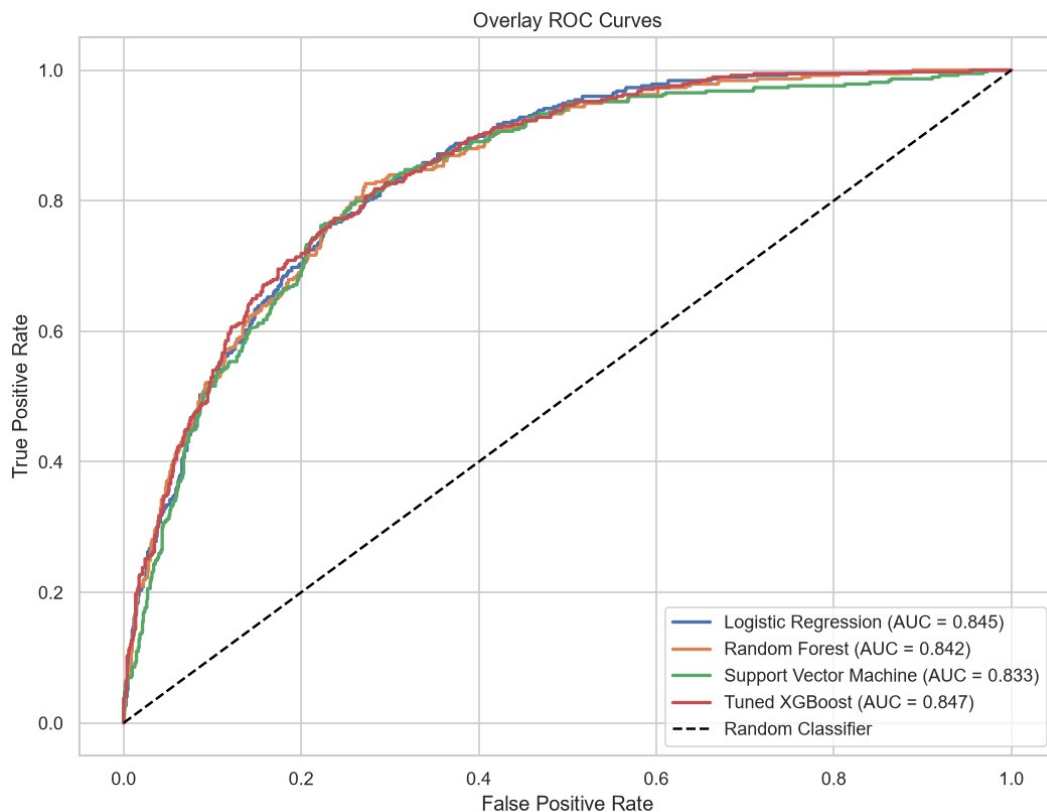


Figure 20: Overlaid ROC curves for the four evaluated models.

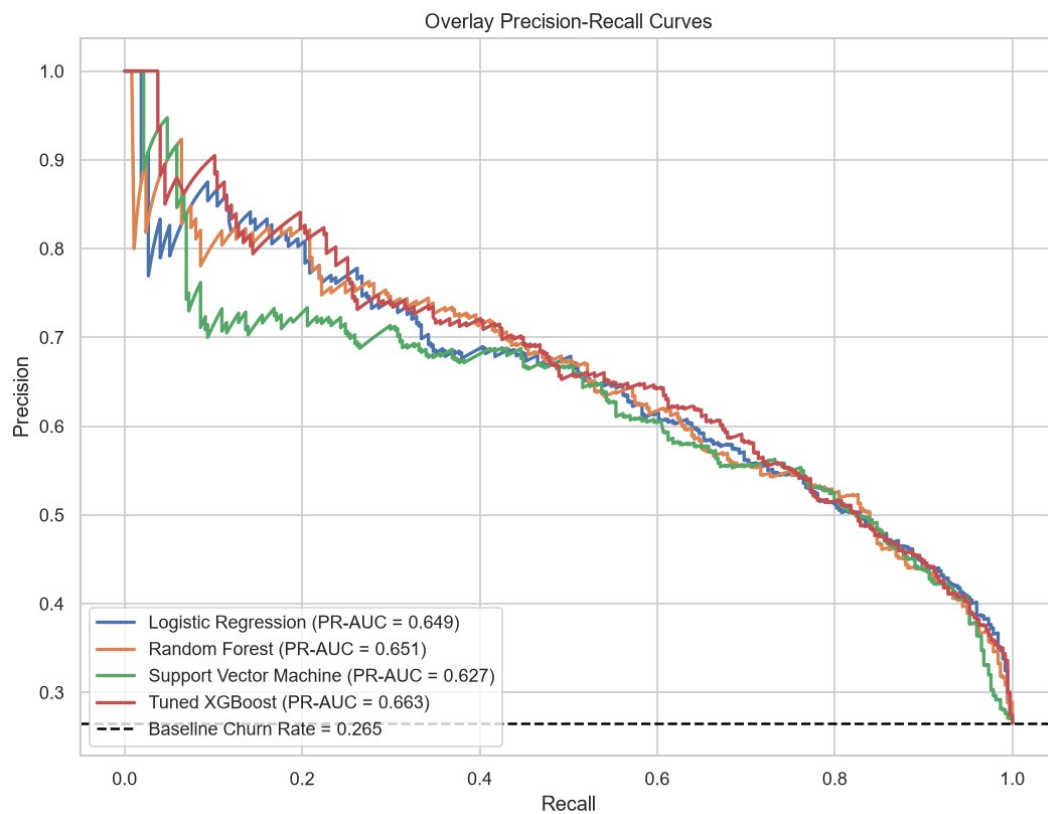


Figure 21: Precision-recall curves. At a 26.54 % base rate, PR-AUC is the more informative summary for an imbalanced problem.

Confusion Matrices at Threshold 0.50

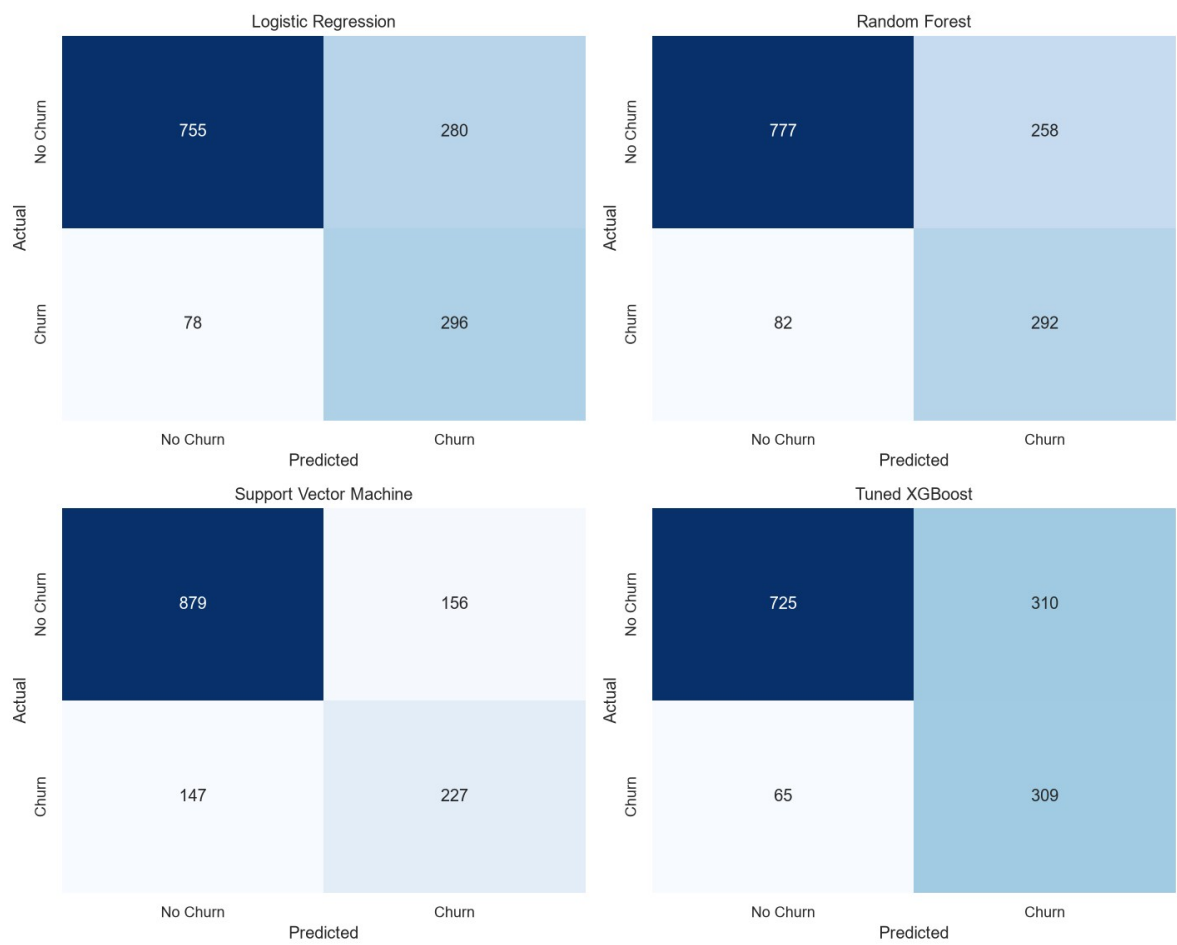


Figure 22: Confusion matrices for all four models side by side.

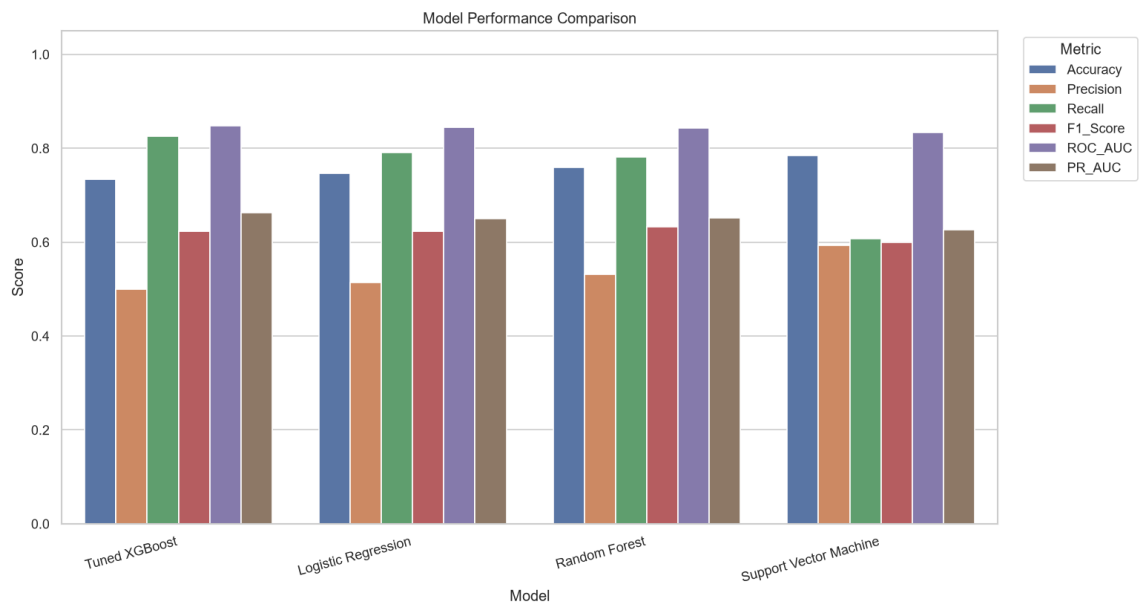


Figure 23: Head-to-head metric comparison across the four models.

17.3 Threshold analysis

The default 0.50 cut-off is a mathematical convention, not a business choice. Because a missed churner costs far more than a false alarm, the economically optimal threshold is almost certainly lower than the point that maximises F1. The sweep makes the trade explicit.

Threshold	Accuracy	Precision	Recall	F1	Reading
0.10	0.4911	0.3419	0.9920	0.5086	catches nearly everyone, flags almost everyone
0.20	0.5791	0.3831	0.9599	0.5477	very aggressive
0.30	0.6373	0.4176	0.9278	0.5759	aggressive
0.40	0.6920	0.4584	0.8850	0.6040	recall-led
0.50	0.7339	0.4992	0.8262	0.6224	default
0.55	0.7466	0.5148	0.7914	0.6238	balanced
0.60	0.7658	0.5415	0.7674	0.6350	balanced
0.65	0.7871	0.5815	0.7059	0.6377	maximum F1; also the phase 9 targeting threshold
0.70	0.7991	0.6207	0.6257	0.6232	precision-led
0.75	0.8020	0.6589	0.5267	0.5854	maximum accuracy
0.85	0.7786	0.7313	0.2620	0.3858	misses three in four
0.90	0.7608	0.8364	0.1230	0.2145	near-useless recall

Table 36: Decision-threshold sweep for the selected model. Abridged from data/phase7_threshold_analysis.csv, which holds 17 rows.

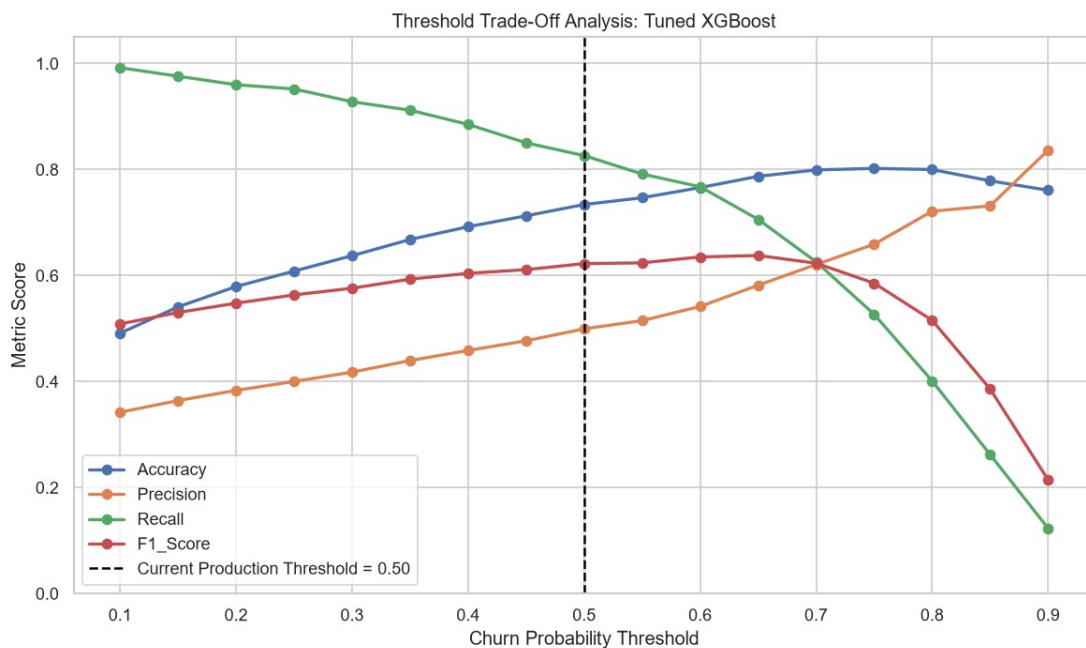


Figure 24: Precision, recall and F1 against the decision threshold.

Accuracy and F1 peak at different thresholds, and neither is the business optimum.

Accuracy is maximised at 0.75, where recall has fallen to 0.5267 and the model misses nearly half of all churners. F1 peaks at 0.65, which happens to coincide with the phase 9 targeting threshold. None of these is derived from the actual cost ratio between a missed churner and a wasted offer, because the project has no campaign-cost data to calibrate against. The sweep is therefore the right artefact to hand a business owner: it lets them choose the operating point once they can price both kinds of error.

18. Phase 8 — Churn Probability and Explainable AI

18.1 Purpose

Phase 8 converts the classifier into a decision tool. It calibrates the predicted probabilities so they can be read as probabilities, assigns every customer a risk tier, computes global feature importance, and produces SHAP explanations at both the global and the individual level. **Modified during remediation** for defects 1 and 2, and in the second round to export a per-customer SHAP driver for every customer.

18.2 Probability calibration

```
calibrated_model = CalibratedClassifierCV(  
    estimator=base_pipeline, method="sigmoid", cv=CALIBRATION_CV_FOLDS)  
calibrated_model.fit(X_train, y_train)
```

Calibration is necessary here precisely because of the class weighting applied in phase 6. Weighting deliberately distorts the predicted scores to improve recall, which means a raw score of 0.7 does not correspond to a 70 % chance of churning. That becomes a problem the moment probability is multiplied by customer value to rank a call list, which is exactly what phase 9 does. Platt scaling fits a sigmoid that maps the distorted scores back onto observed frequencies.

Calibration metric	Value	Reading
Calibrated test ROC-AUC	0.8465	ranking quality preserved (0.8472 uncalibrated)
Brier score	0.1364	mean squared error of the probabilities; lower is better
Mean predicted probability (all customers)	0.2652	against an observed base rate of 0.2654

Table 37: Calibration quality on the hold-out partition.

The agreement between the mean predicted probability (0.2652) and the observed base rate (0.2654) is the clearest single indication that calibration succeeded. Calibration preserves ranking — ROC-AUC moves only from 0.8472 to 0.8465 — while making the absolute values meaningful.

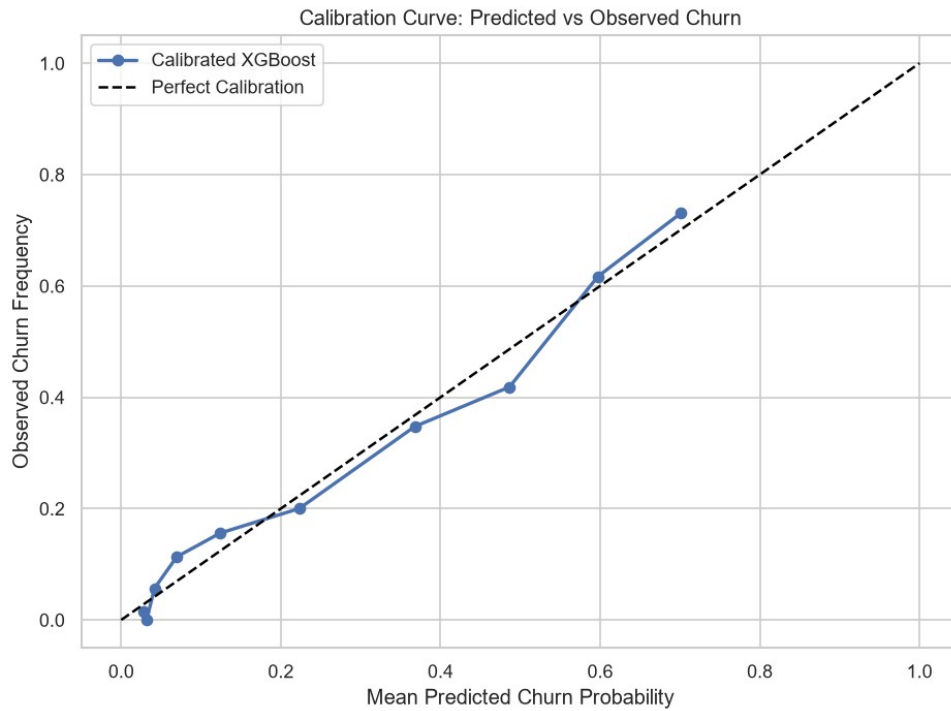


Figure 25: Calibration curve: predicted probability against observed churn frequency, in quantile bins.

18.3 Risk tiering

Risk tier	Probability band	Customers	Share	Mean predicted	Observed churn rate
Low Risk	0.00 – 0.35	4 579	65.02 %	0.1035	9.85 %
Medium Risk	0.35 – 0.65	1 740	24.71 %	0.5099	49.02 %
High Risk	0.65 – 1.00	724	10.28 %	0.7003	78.04 %

Table 38: Risk-tier distribution with the churn rate actually observed in each tier.

The tiers validate cleanly against observed outcomes.

Customers the model places in the High Risk tier churn at 78.04 %; those in the Low Risk tier churn at 9.85 %. That is an eight-fold separation, and the ordering is strictly monotonic across the three tiers. Mean predicted probability also tracks observed frequency closely within each tier (0.7003 against 78.04 %, 0.1035 against 9.85 %), which is the practical payoff of the calibration step.

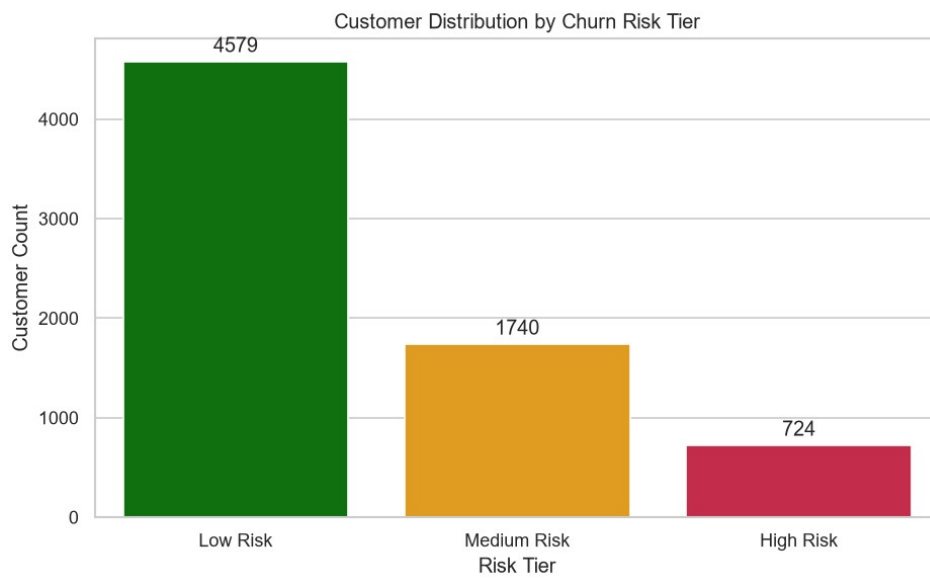


Figure 26: Customer distribution across the three risk tiers.

18.4 Global feature importance

This ranking reflects both remediation rounds. The leaked segment label is gone (round 1), and contract type now appears once, with its importance undivided, because the duplicate **ContractRiskFlag** no longer competes with it (round 2).

Rank	Feature	Gain	Note
1	Contract_Month-to-month	0.2465	sole carrier of the contract signal after round 2
2	Cluster_3	0.1467	the New High-Spend segment; built from attributes only
3	Contract_Two year	0.0783	long commitment protects
4	InternetService_Fiber optic	0.0614	
5	TenureBand_0-6 months	0.0337	recovered by fixing defect 3
6	InternetService_DSL	0.0334	
7	PaymentMethod_Electronic check	0.0275	
8	Contract_One year	0.0269	
9	InternetService_No	0.0267	
10	tenure	0.0183	
17	ValueTier_Medium	0.0124	recovered by fixing defect 3
—	ContractRiskFlag	absent	removed from the matrix in round 2
—	CustomerSegment	absent	removed from the matrix in round 1

Table 39: Top features by gain importance in the phase 8 model after both remediation rounds.

Two movements deserve comment. Contract Month-to-month now carries 0.2465 on its own; previously the same signal was split into 0.2462 and 0.2421 between two identical columns, so neither number described it. And **Cluster_3** has risen to second place. That is legitimate: the cluster is built from tenure, service count and monthly charges only, and after round 1 its name no longer encodes churn. Gain importance is not conserved between models, so when a redundant column is removed the trees redistribute their splits across the remaining features.

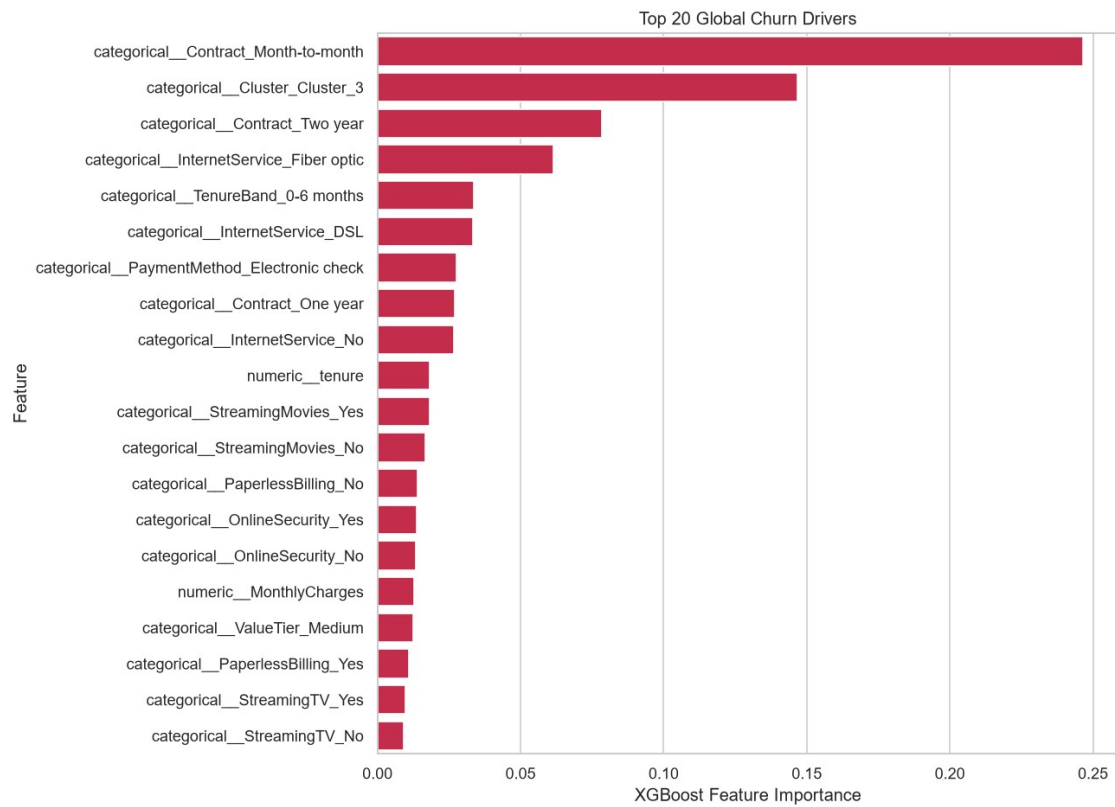


Figure 27: Global feature importance by gain.

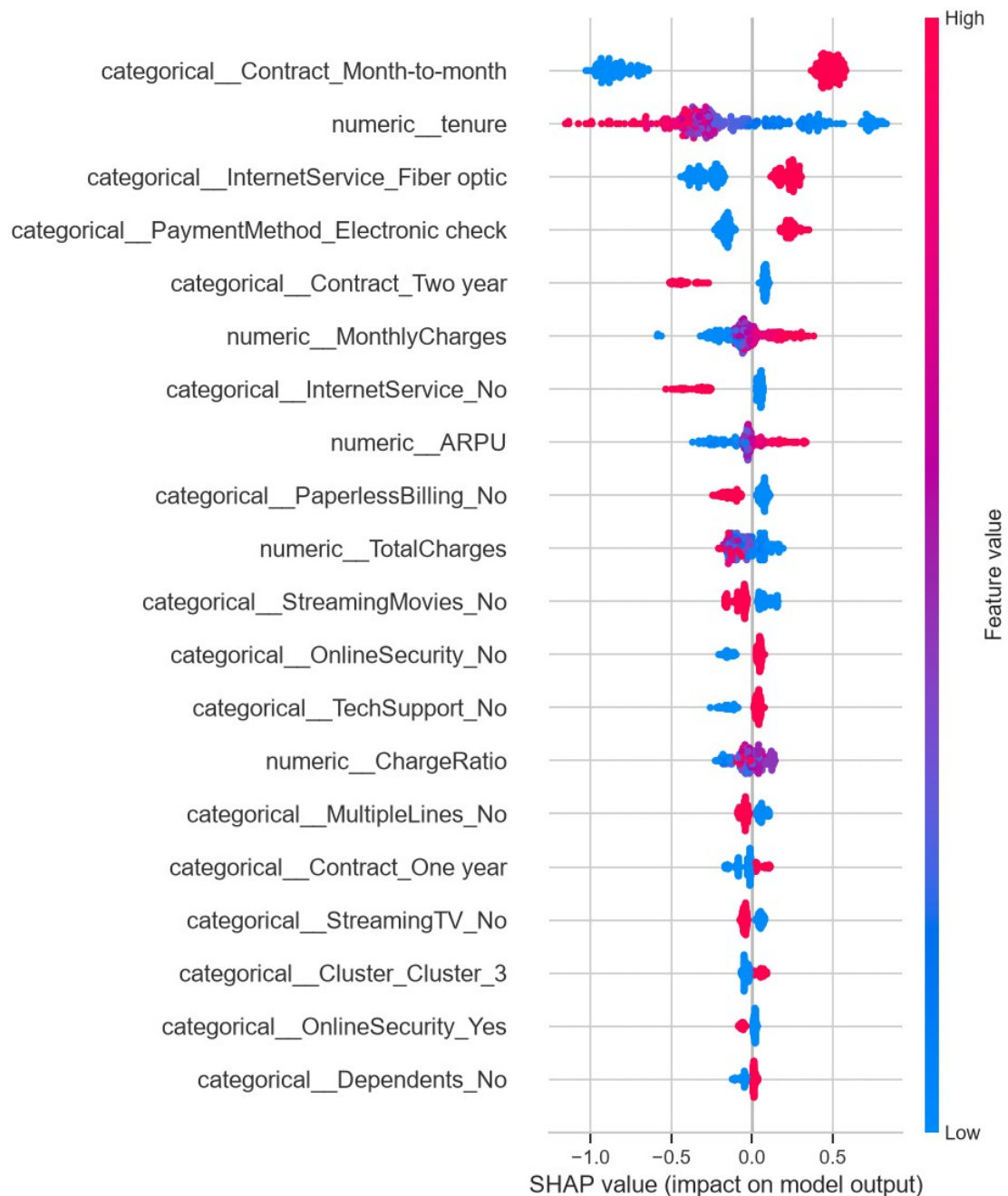


Figure 28: SHAP global summary. Each point is one customer; position shows the direction and magnitude of that feature's contribution.

18.5 Local explanations and reason codes

Global importance says what matters across the base. A retention agent needs to know what matters for the customer in front of them, which is what SHAP provides per prediction. Phase 8 converts the three highest-magnitude contributions per customer into plain reason codes.

customerID	ChurnProbability	RiskTier	ReasonCode1 / 2 / 3
7216-EWTRS	0.7690	High Risk	tenure increased churn risk Contract_Month-to-month increased churn risk InternetService_Fiber optic increased risk
5178-LMXOP	0.7667	High Risk	(same three drivers)
9300-AGZNL	0.7663	High Risk	(same three drivers)
5419-JPRRN	0.7661	High Risk	(same three drivers)
9497-QCMMS	0.7657	High Risk	(same three drivers)

Listing 21: SHAP reason codes for the five highest-risk customers. The contract signal is now cited through the Contract column itself.

All five share the same three drivers because they are the five highest-probability customers in the base, and the strongest signals are contract, tenure and fibre service. That is why phase 9 does not use these raw top-three codes to pick an action: they would repeat contract for almost everyone. Section 18.6 describes what it uses instead.

18.6 Per-customer driver levers

Added in the second remediation round. Phase 8 now computes SHAP values for all 7 043 customers, sums the one-hot columns back to their source feature (the three Contract columns become one Contract contribution), and groups the actionable features into five levers.

Lever	Source features	Action if it is the customer's top lever
Service quality	InternetService	Service-quality audit with proactive fix and loyalty credit
Billing and payment	PaymentMethod, PaperlessBilling, AutoPayFlag	Move to automatic payment with a billing reward
Protection and support	TechSupport, OnlineSecurity, OnlineBackup, DeviceProtection	Complimentary tech-support and online-security trial
Engagement and bundling	StreamingTV, StreamingMovies, ServiceCount, MultipleLines, PhoneService	Bundled add-on offer to deepen product engagement
Price and plan fit	MonthlyCharges, ChargeRatio, ARPU, ValueTier, TotalCharges	Plan right-sizing review and personalised tariff
(shared) Contract	Contract	Discounted 12- or 24-month contract — the base offer for every month-to-month customer

Table 40: Lever groups defined in churn_features.LEVER_GROUPS and their actions.

```
feature_shap = aggregate_shap_by_feature(
    all_shap_values, feature_names, X.columns.tolist())

driver_df = assign_shap_drivers(feature_shap)    # PrimaryLever, SecondaryLever
driver_df.to_csv("data/phase8_customer_risk_drivers.csv", index=False)
```

Listing 22: The per-customer driver export in phase 8.

Tenure, the segment cluster and demographics are deliberately not levers. An offer cannot change how long someone has been a customer or who they are; those attributes tell the business when and to whom to reach out, not what to offer. Among the 724 High Risk customers the primary lever splits as Billing and payment 333, Service quality 273, Price and plan fit 112, Protection and support 4, and Engagement and bundling 2. Every one of them has a lever with a positive contribution.

19. Phase 9 — Business Recommendations and Revenue Impact

19.1 Purpose

Phase 9 converts calibrated probabilities into money. It computes each customer's value and revenue at risk, selects a targeting population, assigns a retention action driven by the customer's dominant risk factor, and models the economics of the resulting campaign. **Modified during remediation** to read recall from phase 7 rather than a hard-coded constant, and in the second round to take each customer's driver from the phase 8 SHAP levers instead of a fixed rule cascade.

19.2 From probability to priority

A churn probability is not by itself a decision. The correct target for retention spend is not the customer most likely to leave, but the one whose departure would cost the most.

Customer Value	=	MonthlyCharges	x	Expected Remaining Tenure
Revenue at Risk	=	P(churn)	x	Customer Value
Target for Retention	=	P(churn) > 0.65		
Expected Saved Rev.	=	Revenue at Risk	x	Campaign Success Rate
Campaign Cost	=	Targeted	x	Cost per Retention Offer
Net Benefit	=	Expected Saved Revenue	-	Campaign Cost
Retention ROI (%)	=	(Net Benefit / Campaign Cost)	x	100

Listing 21: The financial model implemented in phase 9.

19.3 Financial impact

Quantity	Value	Source
Customers in base	7 043	measured
Baseline churn rate	26.54 %	measured from the cleaned data
Model recall	0.8262	read from phase 7 results
High-risk customers targeted	724	p(churn) > 0.65
Average customer value	\$777.14	derived
Total revenue at risk	\$491 820.10	derived
Expected saved revenue	\$147 546.03	at 30 % campaign success
Campaign cost	\$10 860.00	724 offers at \$15.00
Net benefit	\$136 686.03	derived
Retention ROI	1 258.62 %	derived
Expected retained customers	463	derived
Projected churn rate after campaign	19.96 %	from 26.54 % baseline

Table 41: Financial impact summary produced by phase 9 after both remediation rounds.

Three inputs to this model are assumptions, not measurements.

The expected remaining tenure of 12 months, the 30 % campaign success rate and the \$15.00 cost per offer are stated constants at the top of phase 9. The project holds no campaign-outcome data, so none of the three can be validated from the data. Customer counts, churn rates, probabilities and recall are all measured. The calculation shows the shape of the business case and the sensitivity of the return to each assumption; it should be re-run against observed campaign results before any budget decision rests on it.

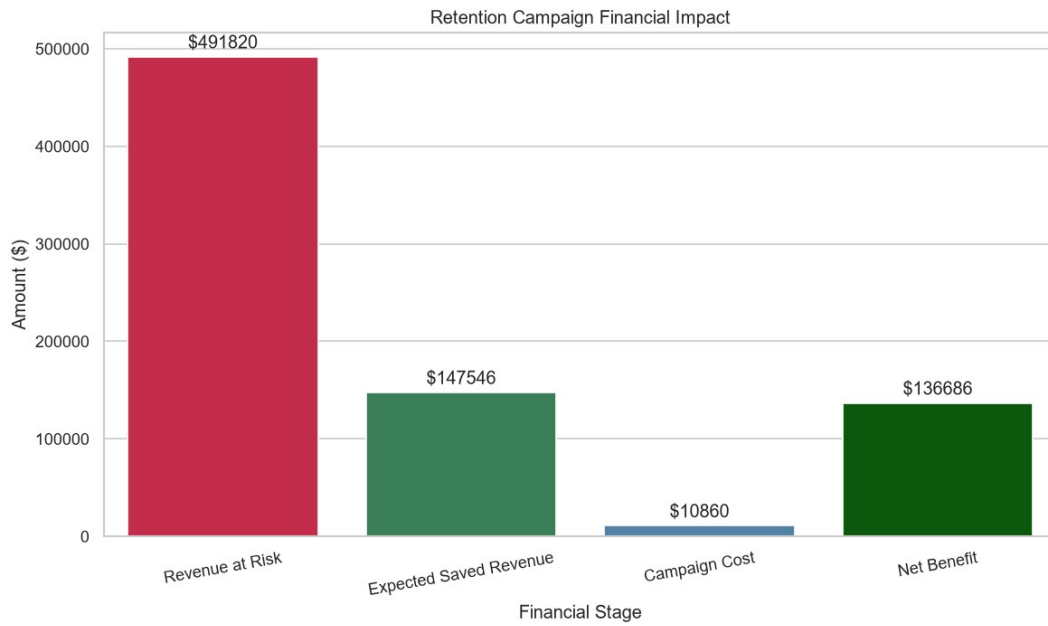


Figure 29: The financial value chain from revenue at risk through to net benefit.

19.4 Where the value sits

Segment	High-risk customers	Revenue at risk	Net benefit	Share of value
New High-Spend	675	\$476 105.69	\$132 706.71	96.8 %
New Budget	48	\$14 830.91	\$3 729.27	3.0 %
Established High-Value	1	\$883.49	\$250.05	0.2 %
Established Budget	0	—	—	—

Table 42: High-risk population and revenue at risk by segment.

The concentration is striking and operationally useful: 96.8 % of the recoverable value sits in a single segment. New High-Spend customers combine the highest churn probability with substantial monthly revenue, so a campaign that addressed only this segment would capture almost all of the available benefit. The Established Budget segment contributes no high-risk customers at all, which is consistent with its 4.44 % churn rate.

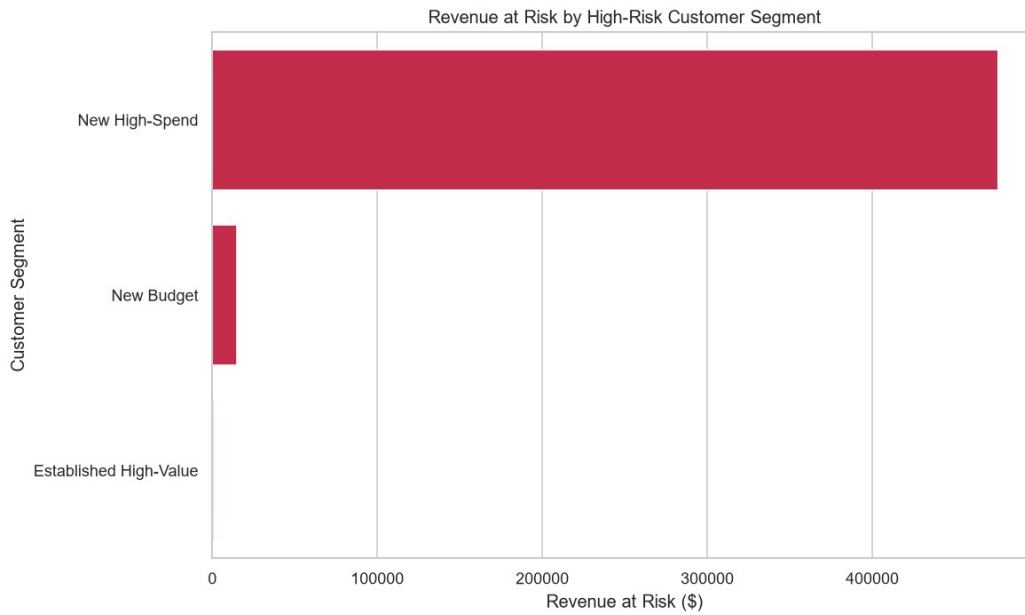


Figure 30: Revenue at risk by high-risk customer segment.

19.5 Driver-to-action mapping

Each targeted customer receives two things: the shared base offer, because all of them are on month-to-month contracts, and a lever-specific action chosen from their own SHAP attribution. The previous rule cascade tested contract first and so put all 732 targeted customers into one category; Section 22.3 records that correction.

```
df["SharedDriver"] = df["Contract"].eq("Month-to-month").map(
    {True: "Month-to-month contract", False: "None"})
df["DominantRiskDriver"] = df["PrimaryLever"] # from phase 8 SHAP

def assign_retention_action(row):
    lever_action = LEVER_ACTIONS.get(row["PrimaryLever"], "Personalised retention review")
    if row["SharedDriver"] == "Month-to-month contract":
        return f"{SHARED_ACTION} + {lever_action}"
    return lever_action
```

Listing 23: Driver and action assignment in phase 9 after the second round.

Dominant driver (SHAP lever)	Lever-specific action	Customers	Revenue at risk	Net benefit
Billing and payment	Move to automatic payment with a billing reward	333	\$207 612.36	\$57 288.71
Service quality	Service-quality audit with proactive fix and loyalty credit	273	\$189 548.51	\$52 769.55
Price and plan fit	Plan right-sizing review and personalised tariff	112	\$93 165.51	\$26 269.65
Protection and support	Complimentary tech-support and online-security trial	4	\$810.58	\$183.17
Engagement and bundling	Bundled add-on offer to deepen product engagement	2	\$683.13	\$174.94

Table 43: Recommendation mapping produced by phase 9. Every row also receives the shared contract offer.

Dominant driver	Onboarding window (0-12 months)	Established (13+ months)
Billing and payment	292	41
Service quality	257	16
Price and plan fit	112	0

Protection and support	4	0
Engagement and bundling	2	0

Table 44: Targeted customers by driver and lifecycle stage. The lifecycle stage tells the team when to reach out; the lever tells them what to offer.

Three levers carry almost all of the value, and they are genuinely different conversations: 333 customers whose risk is driven mainly by billing (manual payment, paper billing), 273 driven by their fibre service, and 112 driven by price relative to what they get. The two small categories are a real result rather than noise to be forced larger: for the highest-risk customers, protection and bundling contribute little to predicted risk. The high-risk list also carries each customer's second lever, so a campaign team can combine offers where the top two contributions are close.

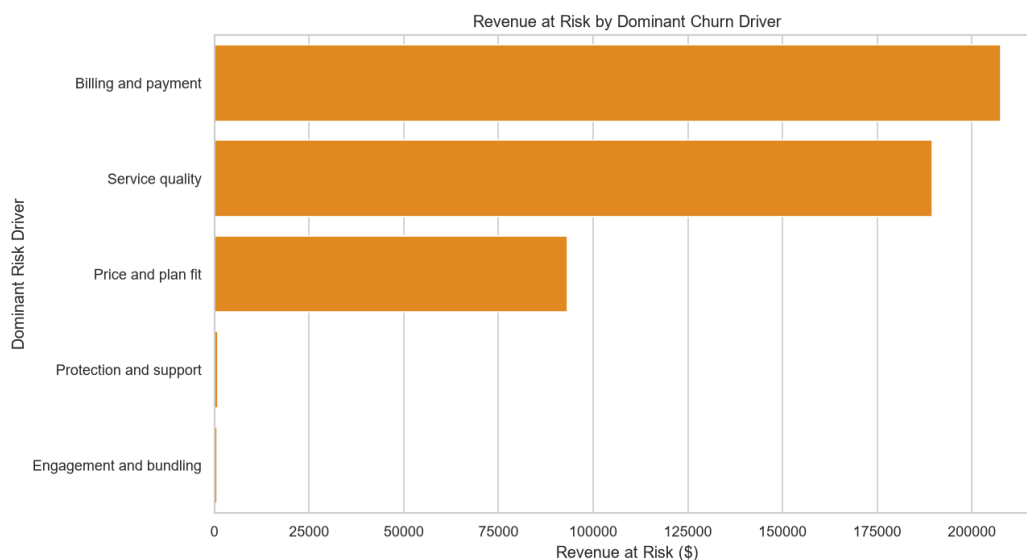


Figure 31: Revenue at risk by dominant churn driver after the second round. The chart now separates five driver categories instead of one.

20. The Streamlit Dashboard

20.1 Purpose and structure

The dashboard is the delivery layer. At 1 145 lines it is the largest single module in the project, and it reads every artefact the nine phases produce. It provides authenticated access, a set of analytical pages, and two scoring routes for customers the model has never seen. **Modified during remediation** so that scoring rebuilds features through the shared module.

Area	Function
Authentication	SQLite-backed accounts with SHA-256 password hashing; login gates every page
Executive overview	Base size, churn rate, revenue at risk and model headline metrics
Exploratory pages	The phase 3 charts with the churn-rate tables behind them
Segmentation	Segment profiles and the phase 5 charts
Model performance	The phase 7 comparison table and evaluation charts
Explainability	Global importance, SHAP summary and per-customer reason codes
Retention targets	The ranked high-risk list, downloadable as CSV
Batch scoring	Upload a CSV and score every row
Single customer	Enter one customer's attributes and score them

Table 45: Functional areas of the dashboard.

20.2 How scoring works after remediation

Both scoring routes now pass through one function, which rebuilds every model input from raw columns using the same code and the same fitted artefacts as the training pipeline:

```
def prepare_scoring_dataframe(uploaded_dataframe):
    scoring_dataframe = uploaded_dataframe.copy()

    feature_artifacts = load_feature_artifacts()      # phase 4 tier edges
    segmentation_model = load_segmentation_model()    # phase 5 scaler+KMeans

    if segmentation_model is None or feature_artifacts is None:
        st.error("Run phases 4 and 5 to generate the required artefacts.")
        st.stop()

    # Engineered features, using the quartile edges fitted during training.
    scoring_dataframe, _ = add_engineered_features(
        scoring_dataframe, artifacts=feature_artifacts)

    # Cluster and segment, assigned by the trained centroids.
    scoring_dataframe = assign_clusters(scoring_dataframe, segmentation_model)

    # The model was trained on the nominal form, so the encoder expects it.
    return to_nominal_cluster(scoring_dataframe)
```

Listing 22: The corrected scoring path. The explicit artefact checks mean a missing model produces a clear instruction rather than a confusing prediction error.

The single-customer page still chooses its recommendation with a short rule cascade (contract, then tech support, then payment method). For one customer that is a single choice rather than a collapse, but it is not the SHAP-lever logic phase 9 now uses; Section 22.4 lists it as the one remaining inconsistency.

The single-customer page previously called the hand-written segment heuristic directly. It now routes through the same function, so a customer entered by hand and the same customer arriving in an uploaded file receive identical feature values and therefore identical predictions.

20.3 Verification

The dashboard was started in the virtual environment and its scoring path exercised against the retrained calibrated model.

Check	Result
Application starts	Yes — no import or load errors
Shared module resolves from <code>app/</code>	Yes — <code>PROJECT_ROOT</code> is already on <code>sys.path</code>
Model input count	26 features
All inputs reconstructed from raw columns	Yes — 0 missing
Engineered features supplied at scoring time	<code>ServiceCount</code> , <code>Cluster</code> , <code>TenureBand</code> , <code>ARPU</code> , <code>ChargeRatio</code> , <code>ValueTier</code> , <code>AutoPayFlag</code>
<code>CustomerSegment</code> passed to the model	No — label only
Batch scoring on 200 held-out rows	succeeded; probabilities 0.0251 to 0.7443, mean 0.2656

Table 46: Dashboard verification after remediation.

Running the dashboard.

From the working directory with the environment active: `streamlit run app/app.py`. The application opens on localhost:8501 and requires an account, which can be created from the login screen. All nine phases must have been executed first, because the dashboard reads their artefacts rather than recomputing anything.

21. Results — Before and After Remediation

21.1 Headline model performance

Metric (Tuned XGBoost)	Delivered	Final	Change	Reading
Test records	353	1 409	4.0×	defect 1
Churners in test set	94	374	4.0×	defect 1
ROC-AUC	0.8349	0.8472	+0.0123	improved
PR-AUC	0.6128	0.6632	+0.0504	the more meaningful gain at this imbalance
Recall	0.7766	0.8262	+0.0496	misses fewer churners
Precision	0.4932	0.4992	+0.0060	essentially unchanged
F1 score	0.6033	0.6224	+0.0191	improved
Accuracy	0.7280	0.7339	+0.0059	essentially unchanged
False negatives	21 of 94 (22.3 %)	65 of 374 (17.4 %)	−4.9 pp	proportionally fewer misses
95 % CI half-width (ROC-AUC)	±0.053	±0.026	halved	defect 1

Table 47: Selected-model performance as delivered and after both remediation rounds.

21.2 Structural changes

Property	Delivered	Final
Model matrix columns	21	26
Segmented dataset columns	25	31
Engineered features in the pipeline	1 of 7	6 of 7 (duplicate flag excluded)
Leaked target information in the matrix	Yes (<i>CustomerSegment</i>)	No
Identical duplicate columns in the matrix	—	None (<i>ContractRiskFlag</i> removed)
Segment names derived from	observed churn rate	centroid geometry
Duplicate segment names	Yes ("New & Fragile 2")	No
Phase 6 vs phase 7 SVM accuracy	0.7601 vs 0.7835	0.7850 vs 0.7850
Distinct phase 9 driver categories	1	5
Serving-time segment assignment	hand-written threshold rule	trained K-Means centroids
Phase 9 recall constant	hard-coded <i>0.7766</i>	read from phase 7
Persisted artefacts	3	5
Environment	none; xgboost missing	dedicated .venv

Table 48: Structural differences between the delivered and the final codebase.

21.3 Effect on the business layer

Quantity	Delivered	Final	Why it changed
Model recall used	0.7766 (hard-coded)	0.8262 (read)	larger hold-out and a better model
High-risk customers targeted	723	724	recalibrated model
Total revenue at risk	\$490 176.80	\$491 820.10	one more customer
Expected saved revenue	\$147 053.04	\$147 546.03	proportional
Campaign cost	\$10 845.00	\$10 860.00	1 more offer

Net benefit	\$136 208.04	\$136 686.03	+\$477.99
Retention ROI	1 255.95 %	1 258.62 %	materially unchanged
Projected churn rate	20.36 %	19.96 %	higher recall retains more customers
Driver / action categories	1	5	issue 6

Table 49: Business-layer outputs as delivered and after both remediation rounds.

The headline business numbers moved only slightly, which is the expected result and worth stating plainly. The targeting threshold and the campaign assumptions did not change, so the population selected is nearly the same. What did change is the standing of the figures: recall is now read from a measurement on a 1 409-record hold-out rather than hard-coded from a 353-record one, so the same conclusion now rests on four times the evidence and cannot silently go stale.

21.4 What the results support

The strongest claim the evidence supports is not the headline ROC-AUC. It is that three methodologically independent analyses agree about what drives churn in this base:

Finding	Exploratory analysis	Segmentation	Model importance
Contract type dominates	42.71 % against 2.83 % across contract levels	implicit in segment separation	Contract_Month-to-month ranks 1st (0.2465)
Early tenure carries the highest risk	52.94 % in months 0-6	both new segments average under 16 months	TenureBand_0-6 ranks 5th
Product bundling protects	5.28 % churn at six add-ons	loyal segment averages 4.16 services	streaming and support features throughout the top 20
Manual payment raises risk	45.29 % on electronic check	not a clustering dimension	ranks 7th; top lever for 333 targets
Demographics do not matter	0.76 pp gender gap	not a clustering dimension	gender absent from the top 20

Table 50: Convergence of three independent methods on the same five findings.

This convergence is what allows the retention strategy to be trusted. Any one method could be misled by an artefact of its own assumptions; three methods with different assumptions reaching the same conclusions is a far stronger position. It is also a conclusion that could not be drawn confidently before remediation, because the model's second-ranked feature was a label derived from the outcome.

21.5 Performance in context

A test ROC-AUC of 0.8472 sits at the upper end of the 0.83 to 0.86 band that published studies report on this dataset. The ceiling is a property of the data rather than the algorithm: the Telco dataset contains no behavioural telemetry — no support-ticket history, no network-quality measurements, no usage trend, no competitor pricing. It describes what a customer buys, not how they experience the service. That the leading models all converge just below 0.85 suggests the feature set, not the modelling, is the binding constraint.

22. Second-Round Corrections

The first revision of this report documented three issues outside the original remediation scope. All three have now been corrected. The affected phases (6, 7, 8 and 9) were re-executed in the virtual environment, and every result elsewhere in this report reflects the corrected code.

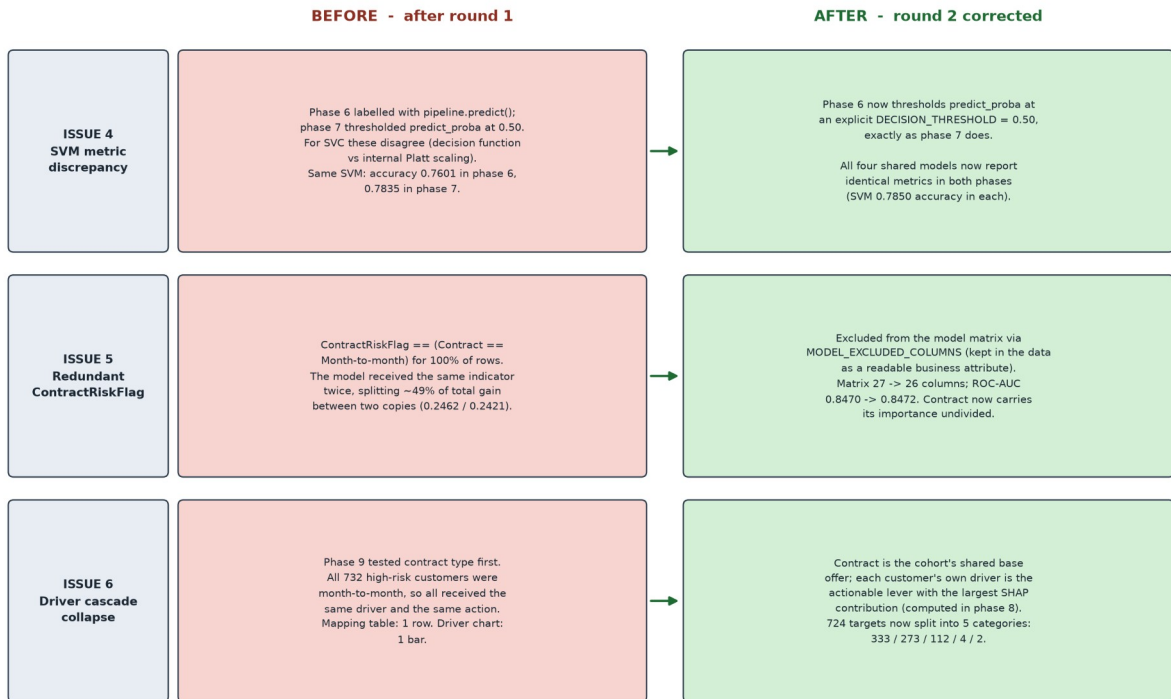


Figure 32: The three second-round issues and their corrections, side by side.

22.1 SVM metric discrepancy between phases 6 and 7

The issue. The two phases published different numbers for the same model on the same split: phase 6 reported the SVM at 0.7601 accuracy and 0.7834 recall, phase 7 at 0.7835 and 0.6150. Phase 6 labelled with `pipeline.predict()`; phase 7 thresholded `predict_proba` at 0.50. For SVC, `predict()` uses the sign of the decision function while `predict_proba` uses Platt scaling fitted by internal cross-validation, and scikit-learn documents that the two can disagree. Running both routes on one fitted SVM gave confusion matrices [778, 257, 81, 293] and [874, 161, 144, 230].

The correction. Phase 6 now labels predictions the same way phase 7 does, from an explicit constant:

```
DECISION_THRESHOLD = 0.50

predicted_probabilities = pipeline.predict_proba(X_test)[: , 1]
predicted_labels = (predicted_probabilities >= DECISION_THRESHOLD).astype(int)
...
best_predictions = (best_probabilities >= DECISION_THRESHOLD).astype(int)
```

Listing 27: Both label assignments in phase 6 now use the explicit threshold.

Model	Phase 6 accuracy	Phase 7 accuracy	Phase 6 recall	Phase 7 recall
Tuned XGBoost	0.7339	0.7339	0.8262	0.8262
Logistic Regression	0.7459	0.7459	0.7914	0.7914
Random Forest	0.7587	0.7587	0.7807	0.7807

Support Vector Machine	0.7850	0.7850	0.6070	0.6070
------------------------	--------	--------	--------	--------

Table 51: After the correction the two phases agree exactly for every shared model.

Verification. No bare `.predict()` call remains in phases 6, 7 or 8. The selected model is unaffected: it is XGBoost, for which the two labelling routes were already equivalent.

22.2 Redundant ContractRiskFlag

The issue. `ContractRiskFlag` equals `Contract == "Month-to-month"` for 100 % of rows, and the one-hot encoding of `Contract` already produces that indicator. The model therefore received the same column twice. The two copies ranked first and second by gain (0.2462 and 0.2421). The model gained no information from the second copy, and a tree ensemble divides importance between identical columns arbitrarily, so both importance numbers were misleading.

The correction. The flag is excluded from the model matrix through the shared exclusion list. It is still computed in phase 4 and kept in the datasets, because as a readable yes/no attribute it is useful in reports and to the business.

```
MODEL_EXCLUDED_COLUMNS = [
    "customerID", "Churn", "ChurnFlag",
    "CustomerSegment", # round 1: leaked label
    "ContractRiskFlag", # round 2: duplicate of Contract
]
```

Listing 28: The exclusion list in `churn_features.py`. Every phase and the dashboard build their matrix through it.

Check	With the flag	Without the flag
Model matrix columns	27	26
Tuned XGBoost hold-out ROC-AUC	0.8470	0.8472
Gain of the contract signal	split 0.2462 / 0.2421	0.2465, one column
Dashboard inputs reconstructed	27 of 27	26 of 26

Table 52: Effect of removing the duplicate flag.

22.3 Driver cascade collapse in phase 9

The issue. Phase 9 picked a driver for each targeted customer with a rule cascade that tested contract type first. All 732 high-risk customers were on month-to-month contracts, so every one of them got the same driver and the same action. The recommendation mapping had one row and the driver chart had one bar, which left nothing to segment an outreach campaign by.

Why a simple SHAP swap would not have fixed it. The obvious fix is to use each customer's top SHAP feature, so that was tested first. Among the high-risk customers it produced only two categories (Contract 463, tenure 269). Excluding Contract still gave only two (tenure 503, InternetService 229). Tenure dominates because high-risk customers are new customers, and an offer cannot change how long someone has been a customer.

The correction. The fix separates what the whole cohort shares from what differs between customers:

1. **Contract is the shared base driver.** Every month-to-month customer receives the contract-migration offer.
2. **Each customer gets a personal lever.** Phase 8 computes SHAP for all customers, sums the one-hot columns back to their source feature and groups them into five actionable levers (Section 18.6). The lever with the largest contribution becomes that customer's driver and adds a lever-specific action.

3. **Lifecycle stage is reported separately.** Tenure decides when to reach out (onboarding window or established), not what to offer.

Driver category	Delivered (cascade)	Final (SHAP lever)
Month-to-month contract	732	— (shared base offer for all 724)
Billing and payment	0	333
Service quality	0	273
Price and plan fit	0	112
Protection and support	0	4
Engagement and bundling	0	2
Distinct categories	1	5

Table 53: Driver distribution among targeted customers before and after the correction.

Verification. The recommendation mapping now has five rows and the driver chart five bars (Section 19.5). Every targeted customer has a lever with a positive SHAP contribution, so no customer falls back to a generic action. Phase 9 checks that the driver file exists and covers every customer, and stops with a clear message if either check fails.

The small categories are left as they are.

Protection and support (4 customers) and Engagement and bundling (2) are small because, for the highest-risk customers, those features add little to predicted risk. Forcing a more even split would mean overriding what the model attributes, which would make the drivers less honest. The high-risk list also carries each customer's second lever, which a campaign team can use when the top two contributions are close.

22.4 Remaining limitations

These are properties of the data rather than defects in the code, but they bound what the system can claim.

- **The single-customer dashboard page still uses a rule cascade.** Batch scoring and phase 9 use the SHAP levers, but the page that scores one hand-entered customer still picks its recommendation by testing contract, tech support and payment method in turn. For one customer this does not collapse anything, but it is inconsistent with phase 9. Loading the SHAP explainer in the dashboard would align it.
- **No temporal dimension.** The dataset is a single cross-sectional snapshot with no observation window, prediction horizon or event date. The model therefore answers "does this customer resemble one who has left" rather than "will this customer leave in the next ninety days". A production system needs the second question, which requires time-stamped records.
- **No behavioural telemetry.** Support-ticket volume, network-quality incidents, usage trend and competitor pricing are all absent. These are typically the strongest churn predictors in telecommunications, and their absence is the most credible explanation for the performance ceiling this dataset consistently produces.
- **ValueTier boundaries are fitted on the full dataset.** Phase 4 computes ARPU quartiles across all 7 043 rows, including those that later become the test set. This is target-independent, so it does not inflate the churn metrics the way defect 2 did, but it is a mild form of information sharing. Fitting the edges on the training partition alone would be stricter.
- **The threshold remains a convention.** Phase 7 sweeps it and phase 9 targets at 0.65, but neither is derived from the actual cost ratio between a missed churning and a wasted offer, because no campaign-cost data exists to calibrate against.

23. Step-by-Step Execution Guide

23.1 Prerequisites

1. Python 3.10 or later on the system path. The pipeline was verified on 3.13.13.
2. Approximately 1.5 GB of free disk space for the virtual environment, which includes XGBoost, SHAP and Streamlit.
3. The raw dataset present at data/Customer-Churn.csv relative to the working directory.
4. At least 4 GB of available RAM. Phases 6 and 7 train ensembles with 2 000 estimators and parallelise across all cores.

23.2 Step 1 — Prepare the environment

```
REM Every script uses relative paths, so this must be the working directory.
cd "C:\Users\user\Desktop\Customer churn prediction\customer_churn_100\customer_churn"

python -m venv .venv
.venv\Scripts\activate

python -m pip install --upgrade pip
python -m pip install -r requirements.txt

REM Verify every import resolves before running any phase
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, xgboost, shap, joblib, streamlit;
print('environment ok')"
```

Listing 23: Environment preparation. The final check must print "environment ok" before proceeding.

23.3 Step 2 — Execute the phases in order

The scripts must run in numerical sequence, because each reads a file the previous one wrote. Note that phase 5 now requires phase 4 to have completed, and will exit with an explanatory message if it has not.

python phase1_data_understanding.py	REM ~2 s	console profile only
python phase2_data_cleaning.py	REM ~3 s	writes Cleaned.csv
python phase3_eda.py	REM ~20 s	writes 9 charts
python phase4_feature_engineering.py	REM ~5 s	writes features + artefacts
python phase5_customer_segmentation.py	REM ~30 s	writes Segmented.csv + model
python phase6_model_development.py	REM ~4 min	trains and tunes 5 models
python phase7_model_evaluation.py	REM ~3 min	evaluation + thresholds
python phase8_churn_probability_xai.py	REM ~1 min	calibration + SHAP
python phase9_business_recommendations.py	REM ~10 s	revenue impact

```
REM Then launch the dashboard
streamlit run app/app.py
```

Listing 24: The complete execution sequence with measured runtimes.

To run the whole pipeline unattended, chain the commands so a failure halts the sequence rather than letting later phases run against stale input:

```
REM PowerShell
foreach ($s in @( "phase1_data_understanding", "phase2_data_cleaning",
                  "phase3_eda", "phase4_feature_engineering",
                  "phase5_customer_segmentation", "phase6_model_development",
                  "phase7_model_evaluation", "phase8_churn_probability_xai",
                  "phase9_business_recommendations" )) {
    Write-Host "=== $s ==="
    python "$s.py"
    if ($LASTEXITCODE -ne 0) { Write-Host "FAILED: $s"; break }
}
```

```
}
```

Listing 25: Unattended execution of the full pipeline.

23.4 Step 3 — Verify the outputs

After phase	Expected artefact	Verification
2	<code>data/Customer-Churn-Cleaned.csv</code>	7 044 lines including header
3	9 files <code>charts/phase3_*.png</code>	all present and non-empty
4	<code>data/Customer-Churn-Features-Unscaled.csv</code>	8 columns, 7 043 rows
4	<code>models/feature_engineering_artifacts.joblib</code>	present — needed by the dashboard
5	<code>data/Customer-Churn-Segmented.csv</code>	31 columns — confirms the merge succeeded
5	<code>models/customer_segmentation_model.joblib</code>	present — needed by the dashboard
6	<code>data/phase6_model_performance.csv</code>	6 lines: header plus 5 models
7	<code>data/phase7_threshold_analysis.csv</code>	18 lines: header plus 17 thresholds
8	<code>data/phase8_customer_churn_risk_predictions.csv</code>	7 044 lines
8	<code>data/phase8_customer_risk_drivers.csv</code>	7 044 lines; needed by phase 9
9	<code>data/phase9_high_risk_retention_targets.csv</code>	725 lines
9	<code>data/phase9_recommendation_mapping.csv</code>	6 lines — one per driver category

Table 54: Artefacts to verify after each phase. The phase 4 and 5 rows are the ones that confirm the remediation is in effect.

23.5 Step 4 — Score new customers

```
import sys, joblib, pandas as pd
sys.path.insert(0, ".")
from churn_features import add_engineered_features, assign_clusters, to_nominal_cluster

model = joblib.load("models/calibrated_churn_probability_model.joblib")
feature_artifacts = joblib.load("models/feature_engineering_artifacts.joblib")
segmentation_model = joblib.load("models/customer_segmentation_model.joblib")

new_customers = pd.read_csv("data/new_customers.csv")
new_customers["TotalCharges"] = pd.to_numeric(
    new_customers["TotalCharges"], errors="coerce").fillna(0.0)

# Rebuild every feature exactly as training did
scored, _ = add_engineered_features(new_customers, artifacts=feature_artifacts)
scored = to_nominal_cluster(assign_clusters(scored, segmentation_model))

scored["ChurnProbability"] = model.predict_proba(
    scored[model.feature_names_in_.tolist()])[:, 1]

# Rank by revenue at risk rather than probability alone
scored["RevenueAtRisk"] = scored["ChurnProbability"] * scored["MonthlyCharges"] * 12
scored.sort_values("RevenueAtRisk", ascending=False).head(200).to_csv(
    "data/retention_priority_list.csv", index=False)
```

Listing 26: Scoring new customers outside the dashboard. Passing the artefacts is what keeps the features consistent with training.

23.6 Troubleshooting

Symptom	Cause	Resolution
<code>ModuleNotFoundError: No module named 'xgboost'</code>	environment not activated	<code>.venv\Scripts\activate</code> , then re-run
<code>SystemExit: Required input not found: ...Features-Unscaled.csv</code>	phase 5 run before phase 4	run phase 4 first — this guard is part of the defect 3 fix
<code>SystemExit: Engineered features are missing for some customers</code>	stale feature file from an earlier dataset	re-run phase 4 to regenerate it

FileNotFoundError: data/Customer-Churn.csv	wrong working directory	cd into customer_churn_100/customer_churn
Dashboard: "Segmentation model not found"	phase 5 has not been run since remediation	run phase 5 to write the segmentation artefact
ModuleNotFoundError: No module named 'churn_features'	script run from outside the working directory	all phases must run with the project folder as cwd
SystemExit: Required input not found: ...risk_drivers.csv	phase 9 run before phase 8	run phase 8 first
Phase 6 appears to hang	150-fit randomized search in progress	expected; takes about 4 minutes. Reduce n_iter to shorten it
InconsistentVersionWarning on load	models pickled by a different scikit-learn	re-run phases 6 and 8 to regenerate them in this environment

Table 55: Common failure symptoms and their resolutions.

24. Conclusion

Six issues have been corrected over two rounds: the three specified defects and the three follow-up issues. Each correction was verified by re-running the pipeline, and the codebase now produces results that can be defended.

24.1 What was fixed

1. **Hold-out sample size.** `TEST_SIZE` moved from 0.05 to 0.20 in phases 6, 7 and 8, enlarging the test set from 353 records to 1 409 and the churner count from 94 to 374. The 95 % confidence interval on ROC-AUC halved, from ± 0.053 to ± 0.026 . The split ratio is now printed from the constant, so the logs cannot again describe a configuration the code does not have.
2. **Target information leakage.** Segment naming now depends only on centroid geometry. The models receive the nominal `Cluster` code and never the churn-derived label, which is absent from the feature-importance ranking of the retrained model. The duplicate segment names were eliminated as a side effect.
3. **Unused engineered features.** Phase 4 exports unscaled features and phase 5 merges them with an integrity check. The engineered features now reach the models, and `TenureBand` ranks 5th by gain.
4. **SVM metric discrepancy.** Phase 6 labels predictions from `predict_proba` at an explicit threshold, so phases 6 and 7 now report identical metrics for every shared model.
5. **Redundant ContractRiskFlag.** Excluded from the model matrix (27 $\rightarrow$ 26 columns) with no loss: ROC-AUC 0.8470 $\rightarrow$ 0.8472.
6. **Driver cascade collapse.** Phase 9 takes each customer's driver from their top SHAP lever, with contract as the shared base offer. The 724 targets now span five driver categories instead of one.

Two consequential corrections were also required. Phase 9 no longer hard-codes the model's recall, and the dashboard no longer approximates the segment label with a hand-written rule — it uses the persisted K-Means model, which removes a train/serve mismatch that existed only because of the leakage.

24.2 What the corrected pipeline achieves

- **Better measured performance on four times the evidence.** ROC-AUC 0.8349 $\rightarrow$ 0.8472, recall 0.7766 $\rightarrow$ 0.8262, PR-AUC 0.6128 $\rightarrow$ 0.6632, all on a hold-out four times larger than before.
- **Risk tiers that validate against outcomes.** 78.04 % observed churn in the High Risk tier against 9.85 % in the Low Risk tier, with strictly monotonic ordering.
- **Probabilities that mean what they say.** Mean predicted probability 0.2652 against an observed base rate of 0.2654, with ranking quality preserved through calibration.
- **Explanations built on legitimate, non-duplicated features.** SHAP explanations no longer rest on a label derived from the outcome, and contract importance is no longer split between two identical columns.
- **Retention actions that differ by customer.** Five driver categories with distinct lever-specific offers, on top of a shared contract-migration base offer.
- **A serving path that matches the training path.** Every model input is rebuilt from raw columns by the same code, with zero missing features at scoring time.

24.3 Recommended next steps

1. **Bring the single-customer dashboard page onto the SHAP levers.** The one remaining place that chooses an action by rule cascade.

2. **Calibrate the decision threshold against real campaign costs.** The threshold sweep is already produced; it needs only a cost ratio to turn into an optimal operating point.
3. **Validate the lever actions with a controlled campaign.** The five actions are reasoned from what drives each customer's risk; a small A/B test per lever would measure which actually retain customers, and replace the 30 % success-rate assumption.
4. **Acquire temporal and behavioural data.** This is the change that would lift performance past the ceiling this dataset imposes, and the only one that would let the system answer the question a production deployment actually asks.

With both rounds of corrections applied, the pipeline has moved from a result that could not be defended in review — a ranking inside sampling noise, built partly on a feature derived from the answer — to one that can. The remaining items extend the system rather than repair it.

A. Appendix A — Configuration Reference

A.1 Path constants

Module	Constant	Value
phase 1	FILE_PATH	data/Customer-Churn.csv
phase 2	INPUT_FILE / OUTPUT_FILE	data/Customer-Churn.csv / data/Customer-Churn-Cleaned.csv
phase 3	INPUT_FILE / OUTPUT_FOLDER	data/Customer-Churn-Cleaned.csv / charts
phase 4	INPUT_FILE	data/Customer-Churn-Cleaned.csv
phase 4	FEATURES_FILE	data/Customer-Churn-Features-Unscaled.csv
phase 4	ARTIFACTS_FILE	models/feature_engineering_artifacts.joblib
phase 5	INPUT_FILE / FEATURES_FILE	data/Customer-Churn-Cleaned.csv / the phase 4 export
phase 5	SEGMENTATION_MODEL_FILE	models/customer_segmentation_model.joblib
phases 6-8	INPUT_FILE	data/Customer-Churn-Segmented.csv
phase 9	INPUT_FILE	data/phase8_customer_churn_risk_predictions.csv
phase 9	EVALUATION_FILE	data/phase7_model_evaluation_results.csv
phase 9	DRIVERS_FILE	data/phase8_customer_risk_drivers.csv

Table 56: Path constants. Entries in bold were added during remediation.

A.2 Analytical constants

Module	Constant	Value	Effect if changed
5-8	RANDOM_STATE	42	changes cluster assignment, the split and every model result
5	OPTIMAL_K	4	alters the number of behavioural segments
6, 7, 8	TEST_SIZE	0.20	size of the hold-out; reverting to 0.05 reinstates defect 1
6	CV_FOLDS	5	lower-variance estimate at higher runtime cost
6	DECISION_THRESHOLD	0.50	label threshold; must match phase 7's DEFAULT_THRESHOLD
9	ONBOARDING_WINDOW_MONTHS	12	boundary of the onboarding lifecycle stage
7	DEFAULT_THRESHOLD	0.50	the operating point at which labels are assigned
8	CALIBRATION_CV_FOLDS	5	folds used by Platt scaling
9	HIGH_RISK_THRESHOLD	0.65	who enters the retention campaign
9	CAMPAIGN_SUCCESS_RATE	0.30	stated assumption
9	COST_PER_RETENTION_OFFER	15.00	stated assumption
9	EXPECTED_REMAINING_TENURE_MONTHS	12	stated assumption

Table 57: Analytical constants and the effect of changing each.

A.3 Model hyper-parameters

Model	Configuration
Logistic Regression	penalty="l2", solver="liblinear", class_weight="balanced", max_iter=2000
Random Forest	n_estimators=2000, max_depth=10, min_samples_split=5, min_samples_leaf=2, class_weight="balanced"
Support Vector Machine	kernel="rbf", C=1.0, gamma="scale", probability=True, class_weight="balanced"
XGBoost baseline	n_estimators=2000, learning_rate=0.10, max_depth=6, subsample=0.85, colsample_bytree=0.85, scale_pos_weight computed

XGBoost tuned	RandomizedSearchCV over 10 parameters, n_iter=30, cv=5, scoring="roc_auc" — 150 fits
Phase 8 XGBoost	n_estimators=200, learning_rate=0.03, max_depth=3, min_child_weight=5, reg_lambda=2, wrapped in CalibratedClassifierCV
K-Means	n_clusters=4, n_init=10, random_state=42, fitted on three standardised dimensions

Table 58: Hyper-parameter configuration of every model in the codebase.

B. Appendix B — Complete Artefact Inventory

B.1 Datasets

File	Shape	Produced by	Consumed by
Customer-Churn.csv	7 043 × 21	supplied	phases 1, 2
Customer-Churn-Cleaned.csv	7 043 × 21	phase 2	phases 3, 4, 5, 9
Customer-Churn-Features-Unscaled.csv	7 043 × 8	phase 4	phase 5 (merged)
Customer-Churn-Model-Ready.csv	7 043 × 36	phase 4	inspection only
Customer-Churn-Segmented.csv	7 043 × 31	phase 5	phases 6, 7, 8, dashboard
phase6_model_performance.csv	5 × 6	phase 6	reporting
phase7_model_evaluation_results.csv	4 × 11	phase 7	phase 9 , dashboard
phase7_threshold_analysis.csv	17 × 5	phase 7	dashboard
phase8_customer_churn_risk_predictions.csv	7 043 × 11	phase 8	phase 9, dashboard
phase8_global_feature_importance.csv	27+ × 2	phase 8	dashboard
phase8_high_risk_customer_reason_codes.csv	5 × 6	phase 8	dashboard
phase8_customer_risk_drivers.csv	7 043 × 10	phase 8	phase 9
phase9_high_risk_retention_targets.csv	724 × 21	phase 9	dashboard
phase9_financial_impact_summary.csv	13 × 2	phase 9	dashboard
phase9_recommendation_mapping.csv	5 × 5	phase 9	dashboard

Table 59: Datasets produced by the pipeline. Bold entries were added during remediation.

B.2 Serialised models

File	Contents	Written by	Used by
best_churn_model.joblib	fitted Pipeline: ColumnTransformer + tuned XGBClassifier	phase 6	dashboard
calibrated_churn_probability_model.joblib	CalibratedClassifierCV wrapping the full pipeline	phase 8	dashboard scoring
feature_engineering_artifacts.joblib	ARPU quartile edges fitted on the training data	phase 4	dashboard
customer_segmentation_model.joblib	fitted StandardScaler, KMeans and the segment name map	phase 5	dashboard
enhanced_churn_model.joblib	unknown — no script in the folder writes this file	unknown	nothing

Table 60: Serialised artefacts. Bold entries were added during remediation.

The last row deserves a note. `enhanced_churn_model.joblib` was present in the delivered project, but no script in the folder produces a file of that name and nothing reads it. Its provenance, training data and performance are unknown. It was left in place because removing files is outside the remediation scope, but an unexplained model artefact in a deliverable folder is a reproducibility hazard: a future reader has no way to tell which of the saved models is authoritative. It should either be regenerated by a documented script or deleted.

B.3 Generated charts

Phase	Charts	Count
-------	--------	-------

Phase 3	univariate analysis; churn by contract, payment method, internet service, senior citizen, tech support, tenure band; correlation heat map; tenure-charges scatter	9
Phase 5	elbow method; silhouette scores; cluster profiles; customer segments scatter	4
Phase 6	ROC curves; confusion matrix; feature importance	3
Phase 7	overlay ROC curves; precision-recall curves; all confusion matrices; model comparison; threshold trade-off	5
Phase 8	calibration curve; risk-tier distribution; global feature importance; SHAP global summary	4
Phase 9	revenue at risk by segment; revenue at risk by driver; retention financial impact	3

Table 61: All 28 charts generated by the pipeline, by phase.

End of report.